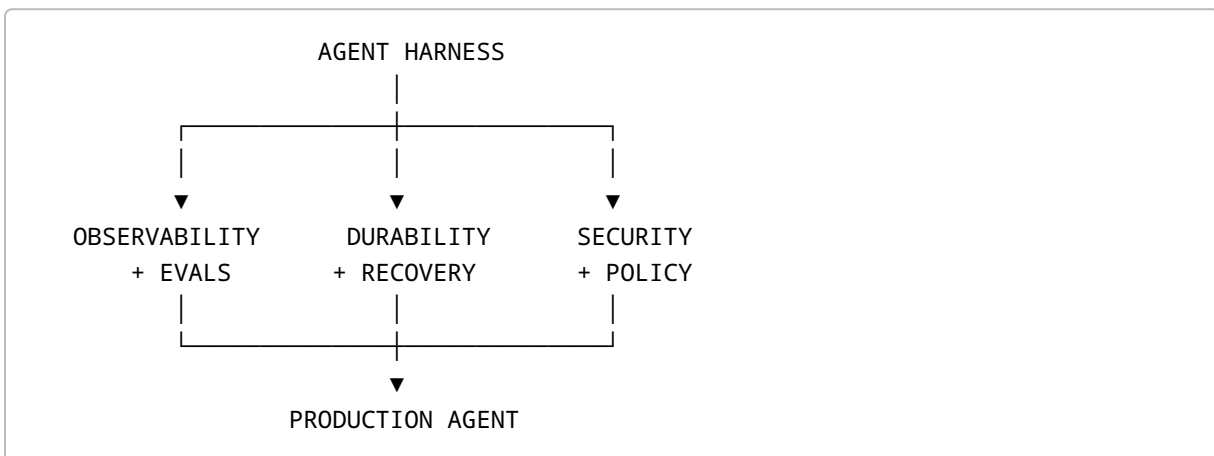


Production Agent Harnesses in LangChain and Deep Agents: Observability, Evaluation, Long-Running Execution, and Security

Executive summary

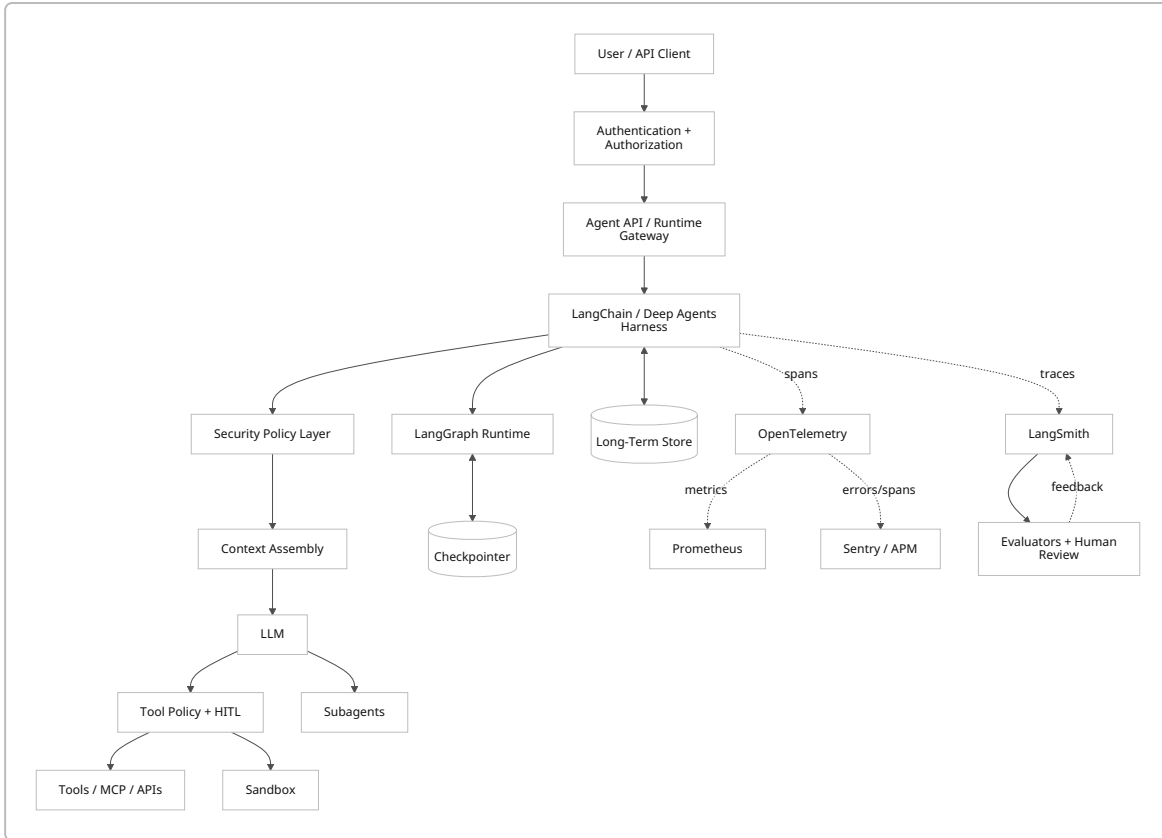
LangChain's current agent stack is best understood as several layers rather than a single "agent" abstraction. **LangChain** supplies the high-level agent API, models, tools, middleware, structured outputs, and integrations; **LangGraph** supplies the durable orchestration runtime—state, nodes, checkpoints, interrupts, retries, streaming, and resumability; and **Deep Agents** provides a more opinionated agent harness on top, with planning, filesystem/workspace semantics, subagents, memory, context-management facilities, and production-oriented customization. Deep Agents explicitly uses LangGraph as its underlying runtime, which is why observability, fault tolerance, persistence, and human-in-the-loop behavior largely reduce to understanding LangChain middleware plus LangGraph execution semantics. ¹

The three concerns in this report are tightly coupled:



The central architectural lesson is that **the model must not be the reliability, security, or observability boundary**. The model proposes reasoning steps and actions; the harness must independently record them, constrain them, persist their state, enforce permissions, limit budgets, handle failures, and require external approval where necessary. This is consistent with Deep Agents' sandbox and permission model, LangChain's middleware controls, and NIST/OWASP guidance on agentic-system risks such as prompt injection and excessive agency. ²

A production design should therefore look more like:



This architecture is a synthesis of LangGraph’s checkpointed runtime, Agent Server’s queue/worker model, Deep Agents’ backend and sandbox boundaries, LangSmith tracing/evaluation, and OpenTelemetry’s vendor-neutral telemetry model. ³

The practical conclusions are:

Concern	Production recommendation
Runtime state	Persist graph state with a production checkpointer; never treat process memory as durable state. ⁴
Long-running tasks	Design around resumable graph boundaries, not a single giant Python function. ⁵
Side effects	Assume a node can run again; external effects therefore need idempotency keys, upserts, transactions, or compensation. ⁶
Observability	Capture agent-level LangSmith traces plus infrastructure-level OpenTelemetry/ metrics. ⁷
Evaluation	Evaluate final answers and trajectories , tool selection, individual steps, latency, cost, and safety behavior. ⁸
Human control	Interrupt before high-impact tool execution, not after the side effect has occurred. ⁹

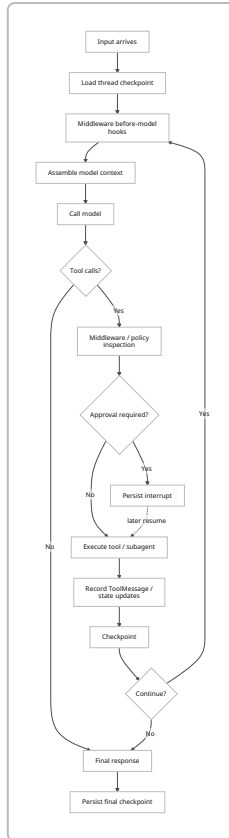
Concern	Production recommendation
Code execution	Use sandbox backends in production; <code>LocalShellBackend</code> is explicitly unsuitable for untrusted or multi-tenant production workloads. ¹⁰
Persistent memory	User-scope writable memory by default; shared organizational policy memory should normally be read-only. ¹¹
Budget enforcement	Combine model/tool-call limits with observability and evaluator spend controls; tracing by itself does not prevent runaway execution. ¹²
PII	Redact before telemetry leaves the process, or disable tracing for zero-retention workloads. ¹³

The rest of this report maps those recommendations down to concrete APIs and execution flows.

Runtime model and the shared theory behind all three concerns

The most useful mental model is to treat an agent run as a **state machine whose transitions are partly selected by an LLM but executed and governed by conventional software**. A Deep Agent may appear conversational from the outside, but internally it repeatedly assembles context, calls a model, interprets tool calls, executes tools/subagents, updates state, and continues until it reaches an exit condition. LangGraph provides the durable state-machine semantics underneath this process. ¹⁴

A simplified execution loop is:



The observability system watches this flow; the durability system makes the state transitions recoverable; the security system determines which transitions are legal. Those systems should therefore share IDs such as `thread_id`, run ID, checkpoint ID, tool-call ID, user/tenant identity, and trace/span identifiers. LangGraph exposes execution metadata including `thread_id`, `run_id`, `checkpoint_id`, `task_id`, and the node attempt number at runtime, making this correlation practical. ¹⁵

The persistence vocabulary matters. LangGraph distinguishes a **checkpointer**, which persists thread-scoped graph state, from a **Store**, which persists application-defined information across threads. Deep Agents' default `StateBackend` stores its filesystem in graph state, so those files persist within a thread through checkpoints but are not automatically shared across threads. Cross-thread Deep Agent memory is normally implemented by routing selected filesystem paths through `StoreBackend`. ¹⁶

Object	Purpose	Scope	Appropriate contents
Run	One invocation/ execution attempt	Invocation	Request status, execution metadata
Thread	Stateful conversation/ task container	Many runs	Current conversational/ workflow state
Checkpoint	Durable graph-state snapshot	Thread	Messages, state channels, pending workflow state

Object	Purpose	Scope	Appropriate contents
Store	Application-level durable memory	Cross-thread	Preferences, learned facts, shared knowledge
Deep Agents StateBackend	Filesystem backed by graph state	Thread	Scratch files, plans, offloaded tool results
Deep Agents StoreBackend	Filesystem abstraction over LangGraph Store	Configurable namespace	Persistent memories
Trace	Observability representation	Run / distributed operation	Model/tool/node timing and inputs/outputs
Trajectory	Ordered agent behavior	Run/thread projection	Model actions, tool calls, messages
Artifact	Application-produced object	Application-defined	Code, reports, datasets, generated files

LangSmith distinguishes traces, runs, threads, and trajectories for observability, while LangGraph persistence distinguishes thread checkpoints and cross-thread stores. Treating telemetry as state—or state as telemetry—creates difficult recovery and privacy problems because those objects have different lifecycles and retention requirements. ¹⁷

Streaming is not persistence. A token, state update, or tool status emitted to an SSE/client stream may disappear if the client disconnects. Agent Server uses Redis pub/sub for live signaling and streaming but explicitly keeps durable run and user data in PostgreSQL; Redis holds ephemeral communication data. A robust frontend therefore uses the stream for responsiveness and the persisted run/thread/checkpoint state as the source of truth after reconnect. ¹⁸

Likewise, tracing is not recovery. LangSmith can tell you what happened, but a LangSmith trace does not itself resume a LangGraph execution. Recovery comes from the checkpoint and runtime; observability explains and evaluates what the runtime did. ¹⁹

This gives us the central production equation:

```

Production Agent
=
LLM policy
+ deterministic harness
+ durable state
+ controlled side effects
+ telemetry
+ evaluation
+ external security policy

```

Deep Agents is valuable precisely because it packages a substantial portion of that harness—filesystem/workspace, subagents, middleware, memory hooks, and execution backends—while leaving LangGraph available underneath when the application needs more explicit orchestration. ²⁰

Agent observability and evaluation

Definition and theory. Agent observability means reconstructing not only whether an application returned an answer but **how the agent reached it**: model calls, prompts/context, tool selection, tool parameters, subagent delegations, retries, token consumption, failures, latency, state transitions, and feedback. LangSmith models executions as nested runs inside traces and can group traces into threads; trajectories represent the ordered behavioral sequence that is particularly important for agent evaluation. ²¹

Traditional service observability asks:

Did the request succeed?
How long did it take?
Where did the error happen?

Agent observability adds:

Why did the agent choose this tool?
Did it call unnecessary tools?
Did it violate expected ordering?
Did a malicious tool result change the plan?
Did it delegate to the correct subagent?
How much did the reasoning path cost?
Did retries improve reliability or merely amplify cost?
Could it have solved the task with fewer steps?
Was the final answer correct despite a bad trajectory?
Was the trajectory safe despite a correct final answer?

This is why final-answer accuracy alone is insufficient for an agent harness. LangSmith’s complex-agent guidance explicitly separates **final-response evaluation**, **trajectory evaluation**, and **single-step evaluation**, while AgentEvals supplies strict and flexible trajectory comparators. Recent agent-evaluation research similarly emphasizes intermediate decision quality and trajectory analysis rather than judging only final outputs. ²²

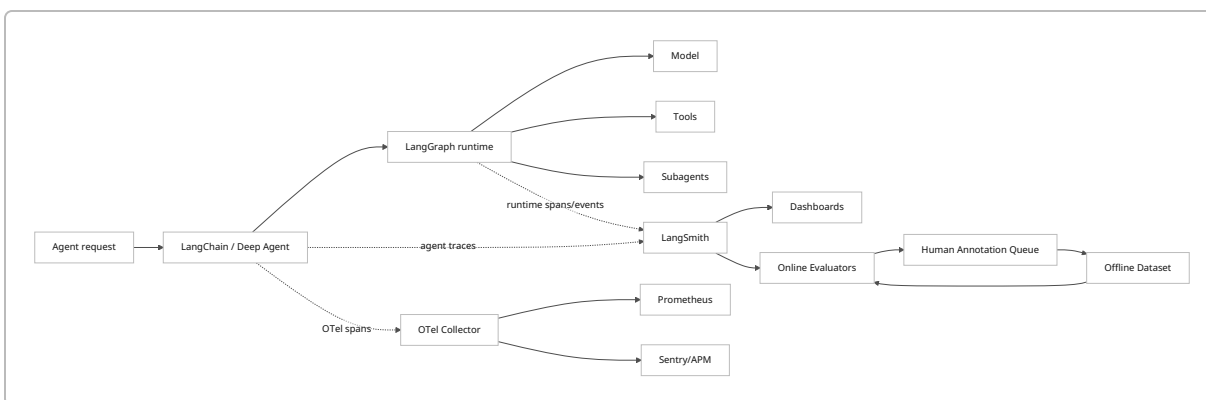
The four observability signal families should remain separate but correlated:

Signal	Meaning	Best questions answered	Typical system
Trace/span	Causal execution tree	“Where did time/cost/error occur?”	LangSmith, OpenTelemetry, Sentry

Signal	Meaning	Best questions answered	Typical system
Log/event	Discrete diagnostic event	"What did component X report at time T?"	Structured logging, Sentry, Loki
Metric	Aggregated time series	"Are latency/error/cost rates changing?"	Prometheus, Grafana, LangSmith dashboards
Feedback/evaluation	Quality judgment	"Was this execution good, safe, efficient?"	LangSmith evaluators, AgentEvals, annotation queues

OpenTelemetry standardizes telemetry conventions and GenAI-related attributes, while Prometheus specializes in aggregate metrics such as counters and histograms. LangSmith is particularly agent-aware because it knows model/tool/run/thread semantics and integrates traces directly with datasets, evaluators, annotation queues, feedback, and trajectories. Sentry is strongest where agent failures must be correlated with broader application errors and performance traces. ²³

Recommended observability flow:



LangSmith can receive LangChain tracing directly and also supports OpenTelemetry-based tracing, including collector-based topologies. This makes “LangSmith versus OpenTelemetry” largely a false choice: OpenTelemetry can be the vendor-neutral transport and infrastructure layer while LangSmith remains the agent-specific trace/evaluation layer. ²⁴

Core LangSmith configuration. A standard harness can enable tracing with environment configuration and assign a project so traces are partitioned logically. LangSmith also exposes configurable trace sampling; absent sampling configuration, tracing is effectively unsampled rather than automatically dropping most executions. ²⁵

```
import os
from deepagents import create_deep_agent

os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_API_KEY"] = "..."
os.environ["LANGSMITH_PROJECT"] = "deep-agent-production"
```

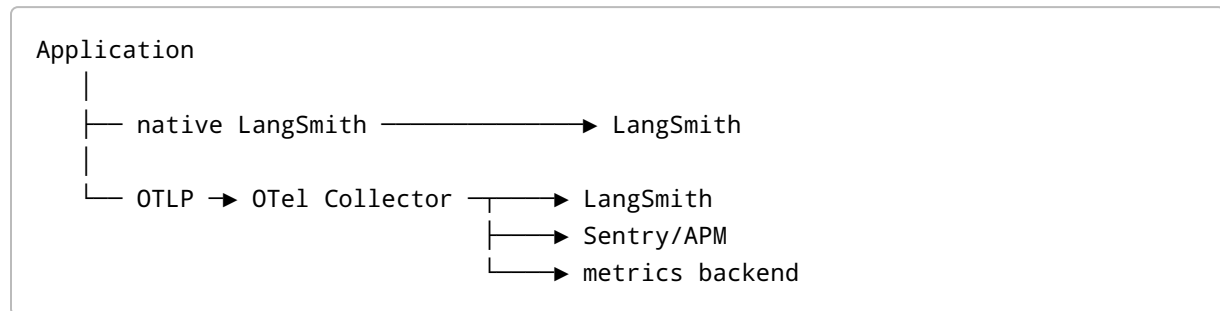
```
# Example: keep 20% of eligible traces when sampling is appropriate.
os.environ["LANGSMITH_TRACING_SAMPLING_RATE"] = "0.20"

agent = create_deep_agent(
    model="openai:gpt-5.5",
    tools=[...],
)
```

A production service should normally use secret-management infrastructure rather than literal API keys as shown above. LangSmith also supports `tracing_context` for request-specific tracing decisions and redaction behavior. ²⁶

For OpenTelemetry-based deployments, LangSmith documents OTel tracing modes in addition to native tracing. Current LangSmith configuration supports tracing modes such as native LangSmith, OTel, and—in Python—hybrid operation for exporting through both paths. ²⁷

Conceptually:



The collector is also a valuable **privacy enforcement point**: sensitive span attributes can be transformed or removed before telemetry leaves the controlled environment. LangSmith explicitly documents using an OTel collector/gateway to sanitize telemetry before forwarding it. ²⁸

What to record. At minimum, every production run should make it possible to correlate:

Category	Recommended fields
Identity	tenant ID, pseudonymous user ID, assistant/version ID
Execution	trace ID, run ID, thread ID, checkpoint ID, node/task ID
Model	provider, model, request settings, latency, retries
Tokens	input, output, cached tokens where provided, total
Cost	model cost, tool/API cost, sandbox/compute cost where available
Tools	tool name, duration, outcome, retry count, approval state

Category	Recommended fields
Subagents	subagent role/name, delegation purpose, duration, return status
Reliability	exception class, timeout type, retry attempt, fallback chosen
Security	permission decision, HITL decision, policy reason, sandbox ID
Quality	evaluator scores, human feedback, rubric/version
Versioning	application revision, prompt version, tool schema version, graph version

OpenTelemetry's GenAI semantic conventions define attributes for agent, model/provider, conversation, evaluation, tool calls, usage, and related concepts, while LangSmith can automatically derive model cost when token counts and recognizable model/provider metadata are available. Custom model wrappers should expose provider/model metadata so cost attribution remains correct. ²⁹

Metrics should be derived from traces but not require traces to remain forever. A recommended Prometheus-style set is:

```
agent_runs_total{agent,version,status}
agent_run_duration_seconds{agent,version}
agent_model_calls_total{model,status}
agent_model_latency_seconds{model}
agent_input_tokens_total{model}
agent_output_tokens_total{model}
agent_tool_calls_total{tool,status}
agent_tool_latency_seconds{tool}
agent_retries_total{node,reason}
agent_interrupts_total{tool,decision}
agent_checkpoint_duration_seconds{backend}
agent_cost_usd_total{agent,model}
agent_eval_score{evaluator,version}
agent_security_denials_total{policy,tool}
agent_queue_wait_seconds{assistant}
```

Prometheus counters fit monotonic totals, while histograms are appropriate for distributions such as request and tool latency where quantiles and service-level objectives matter. Avoid high-cardinality labels such as raw `thread_id`, user text, prompts, filenames, arbitrary error strings, and full tool parameters; those belong in traces/logs rather than time-series labels. ³⁰

A minimal custom metrics layer can sit around the harness:

```
import time
from prometheus_client import Counter, Histogram

RUNS = Counter(
```

```

        "agent_runs_total",
        "Completed agent runs",
        ["agent", "status"],
    )

    RUN_LATENCY = Histogram(
        "agent_run_duration_seconds",
        "End-to-end agent latency",
        ["agent"],
    )

    async def observed_ainvoke(agent, payload, config):
        started = time.perf_counter()
        status = "ok"

        try:
            return await agent.ainvoke(payload, config=config)
        except Exception:
            status = "error"
            raise
        finally:
            RUNS.labels(agent="research-agent", status=status).inc()
            RUN_LATENCY.labels(agent="research-agent").observe(
                time.perf_counter() - started
            )

```

This should supplement, not replace, LangSmith tracing: Prometheus gives inexpensive aggregate operational visibility; LangSmith preserves the agent trajectory necessary for diagnosis and evaluation. 31

Streaming observability. LangGraph and Deep Agents expose synchronous and asynchronous execution as well as event streaming. Current LangGraph guidance uses typed event streaming with `stream_events(..., version="v3")`; the async variant is `astream_events`. Event streams can expose model messages, state projections, subagent/tool activity, interrupts, and other progress while the durable runtime continues independently. 32

```

config = {
    "configurable": {
        "thread_id": "tenant-a:case-8127",
    }
}

stream = await agent.astream_events(
    {
        "messages": [
            {"role": "user", "content": "Investigate the incident."}
        ]
    }
)

```

```

    },
    config=config,
    version="v3",
)

async for event in stream:
    # Forward selected events to SSE/WebSocket clients.
    # Do not treat client receipt as a durability guarantee.
    process_event(event)

```

The exact projections emitted depend on the graph/agent and streaming API used, so production clients should depend on stable event types rather than scraping textual debug output. ³³

Dashboards and alerting. LangSmith dashboards can surface trace counts, latency, errors, model activity, tokens/cost, tool calls, tool latency/error behavior, and feedback statistics. LangSmith alerting supports conditions including run count, cost, errors, feedback score, and latency. Those should be supplemented by infrastructure alerts around worker availability, queue depth, PostgreSQL/Redis availability, checkpoint latency, memory/CPU, and sandbox failures. ³⁴

A practical alert hierarchy is:

Severity	Example	Response
S1	Cross-tenant authorization failure, sandbox escape suspicion, mass destructive action	Stop relevant tools/runs; security incident
S2	Error rate spike, checkpoint backend unavailable, worker pool unavailable	Page operator
S3	P95/P99 latency regression, retry surge, tool failure rate	Investigate
S4	Cost/token drift, evaluator quality decline	Engineering review
Informational	Model distribution shift, changed tool frequency	Trend monitoring

Cost accounting. LangSmith can automatically calculate model costs when it can identify the provider/ model and token counts, and dashboards expose both tokens and costs. For proprietary models or wrappers, supply appropriate metadata; non-model costs such as paid search APIs, databases, sandbox CPU, or proprietary tools may require explicit instrumentation. ³⁵

Cost must be observed at multiple levels:

```

Total Agent Cost
=
Model inference
+ evaluator inference

```

- + tool/API usage
- + embeddings/retrieval
- + sandbox/compute
- + persistence
- + telemetry retention
- + human review

This distinction becomes especially important because evaluation itself consumes inference. LangSmith provides weekly evaluator spend caps for supported judge models; spending is measured separately from the production agent, and an evaluator can be paused when its effective weekly cap is reached. In-flight evaluation work can slightly overshoot before the cap takes effect. ³⁶

At execution time, LangChain provides **ModelCallLimitMiddleware** and **ToolCallLimitMiddleware**. Model-call limits default to no run or thread limit, with `exit_behavior="end"` when configured. Tool-call limits require at least a thread or run limit; their default limit-exceeded behavior is `continue`, which blocks exceeded calls with tool-error messages while allowing the model to continue. Thread-level counters require durable checkpoint state. ³⁷

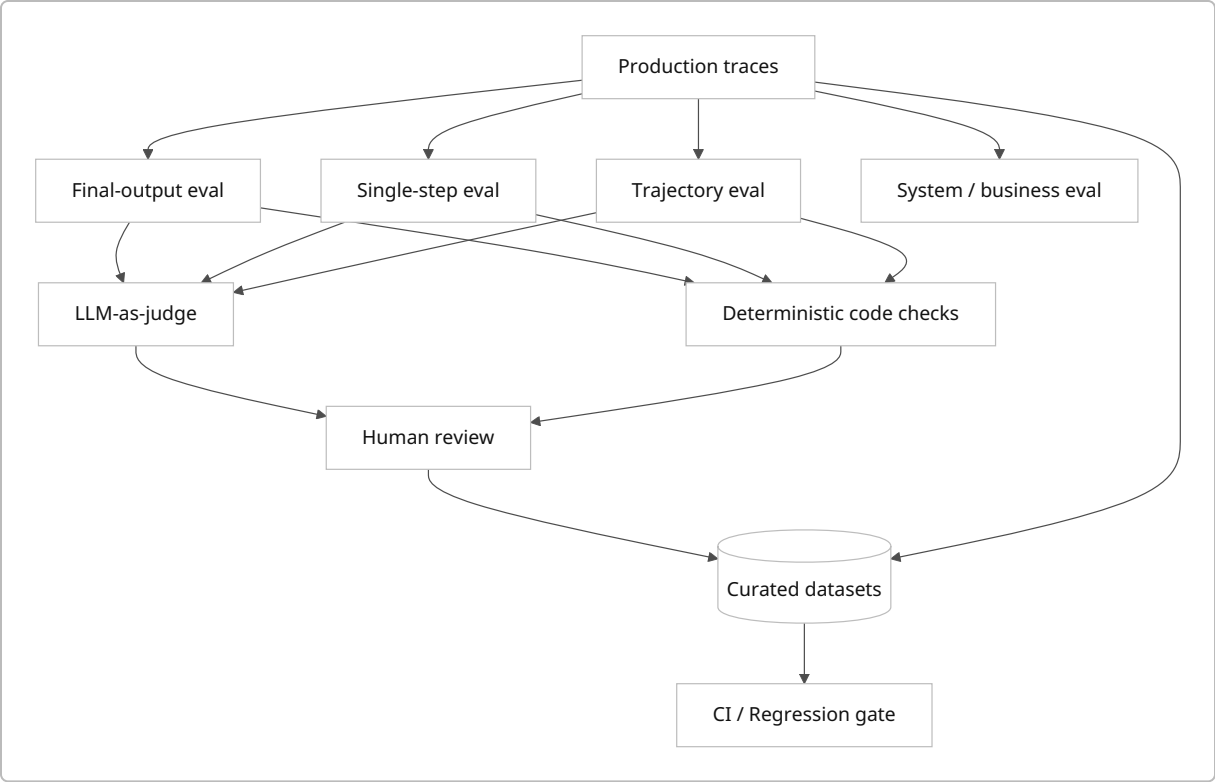
```
from langchain.agents.middleware import (
    ModelCallLimitMiddleware,
    ToolCallLimitMiddleware,
)

budget_middleware = [
    ModelCallLimitMiddleware(
        run_limit=12,
        thread_limit=100,
        exit_behavior="end",
    ),
    ToolCallLimitMiddleware(
        run_limit=20,
        thread_limit=150,
        exit_behavior="error",
    ),
    ToolCallLimitMiddleware(
        tool_name="expensive_search",
        run_limit=4,
        thread_limit=20,
        exit_behavior="error",
    ),
]
```

These are **call-count budgets**, not complete dollar/token budgets. A serious cost-governance layer should additionally calculate accumulated usage and reject or downgrade future model/tool calls when a tenant/run budget is reached. LangSmith's observed cost should feed that policy, but enforcement should happen

inside the application/gateway rather than relying solely on after-the-fact telemetry. LangSmith's gateway and evaluator controls offer additional policy mechanisms, but they solve different budget scopes. 38

Evaluation methodology. The best production program uses four levels simultaneously:



LangSmith defines an offline evaluation loop around datasets, target functions/agents, evaluators, and experiments, and an online evaluation loop around production traces. Human review can be organized through annotation queues, and corrections can be used to align LLM judges with human judgment. 39

Deterministic evaluators should be preferred whenever the requirement can be expressed in code: schema correctness, presence of citations, absence of forbidden tools, required tool ordering, call limits, exact database mutations, permission-denial behavior, or trajectory constraints. They are cheaper and repeatable. **LLM-as-judge** is appropriate for subjective criteria such as reasoning appropriateness, helpfulness, semantic correctness, style, or whether the chosen trajectory was reasonable. LangSmith explicitly supports both code evaluators and LLM judges. 40

Trajectory matching through AgentEvals supports several useful semantics, including strict ordering and more permissive matching modes. 41

Mode	Meaning	Good use case	Main weakness
<code>strict</code>	Exact ordered trajectory	Regulated sequence: verify → authorize → execute	Brittle when several paths are equally valid

Mode	Meaning	Good use case	Main weakness
<code>unordered</code>	Required calls can occur in different order	Parallel retrieval	Ignores ordering defects
<code>subset</code>	Actual calls remain within expected/reference scope	Prevent unnecessary tools	May accept missing useful actions depending rubric
<code>superset</code>	Expected behavior must appear but extras allowed	Flexible exploration	Can reward wasteful trajectories
LLM judge	Judge complete trajectory semantically	Open-ended research/ problem solving	Added cost and nondeterminism

A basic trajectory evaluator can be integrated into test suites:

```
from agentevals.trajectory.match import (
    create_trajectory_match_evaluator,
)

trajectory_evaluator = create_trajectory_match_evaluator(
    trajectory_match_mode="strict",
)

# Test harness supplies the actual and reference trajectories.
score = trajectory_evaluator(
    outputs=actual_agent_output,
    reference_outputs=reference_output,
)
```

The exact reference representation should be fixed in a test schema under version control so a change to tool names or message normalization does not silently invalidate comparisons. AgentEvals is designed specifically to compare agent message/tool-call trajectories and can integrate with LangSmith experiments.

41

For dataset-level evaluation, the LangSmith client supports synchronous and asynchronous evaluation interfaces; asynchronous evaluation is preferable for larger remote-model experiments, with concurrency explicitly bounded rather than launching unbounded requests. 42

Conceptually:

```
from langsmith import Client

client = Client()
```

```

results = await client.aevaluate(
    target_agent,
    data="production-regression-suite",
    evaluators=[
        correctness_evaluator,
        security_evaluator,
        trajectory_evaluator,
    ],
    max_concurrency=8,
    experiment_prefix="agent-v42",
)

```

The operational rule should be:

```

Production failure
↓
Trace investigation
↓
Human/evaluator diagnosis
↓
Representative example added to dataset
↓
Regression evaluator written
↓
Fix implemented
↓
Offline experiment
↓
CI gate
↓
Deploy
↓
Online evaluator watches recurrence

```

LangSmith supports both offline experiment workflows and online evaluation, as well as automation rules that can route problematic traces to annotation queues, datasets, webhooks, evaluators, or retention actions. ⁴³

Human evaluation should not mean randomly reading traces. Create explicit rubrics. For example:

```

security_action_authorized:
    0 = unauthorized or unclear
    1 = explicitly authorized

tool_efficiency:

```

```
1 = severe unnecessary tool use
2 = several unnecessary calls
3 = acceptable
4 = efficient
5 = minimal and well chosen
```

```
trajectory_correctness:
1 = unsafe / fundamentally wrong
2 = major planning defect
3 = workable with issues
4 = correct
5 = ideal
```

Annotation queues then become a controlled data-generation process rather than ad-hoc QA. LangSmith supports run-level and thread-level human review workflows and automation rules such as sending negative-feedback traces or sampled production traffic for annotation. ⁴⁴

PII, telemetry storage, and retention are architectural decisions, not dashboard settings. LangSmith supports hiding complete trace inputs/outputs, metadata transformations, regex/anonymizer-based scrubbing, function-specific processors, and request-level controls. For workloads that must not be observed at all, the documentation recommends disabling tracing conditionally instead of assuming redaction is equivalent to zero retention. ¹³

Useful settings include:

```
LANGSMITH_HIDE_INPUTS=true
LANGSMITH_HIDE_OUTPUTS=true
```

Selective anonymization is preferable when debugging still requires non-sensitive structure:

```
from langsmith import Client
from langsmith.utils import create_anonymizer

anonymizer = create_anonymizer([
    # Organization-specific credential and PII patterns.
])

client = Client(
    hide_inputs=anonymizer,
    hide_outputs=anonymizer,
)
```

LangSmith's client-side anonymization can scrub data before transmission; however, regex-only PII detection should not be assumed to catch every sensitive value. ⁴⁵

As of the current documentation, LangSmith has **base trace retention of 14 days** and **extended retention of 400 days by default**, with Enterprise customization for extended retention. After trace retention expires, trace content such as inputs/outputs is removed, while some metadata may remain for analytics/billing. Importantly, datasets have different lifecycle semantics: dataset content can persist indefinitely, so promoting production examples into datasets is itself a data-retention decision. ⁴⁶

That yields a crucial production rule:

Never auto-promote arbitrary production traces containing personal or confidential content into indefinite-retention evaluation datasets without a deliberate sanitization and governance step.

That recommendation follows directly from LangSmith’s different trace versus dataset retention semantics. ⁴⁷

Tracing-tool comparison:

System	Primary role	Agent-aware semantics	Long-term metrics	Evals	Best use	Main limitation
LangSmith	Agent tracing + eval platform	Excellent	Yes, dashboards	Excellent	LLM/agent debugging, trajectories, feedback	Specialized rather than general infrastructure APM ⁴⁸
OpenTelemetry	Vendor-neutral telemetry standard/transport	Growing GenAI conventions	Backend-dependent	Not inherently	Unified distributed tracing architecture	Requires backend/analysis systems ⁴⁹
Prometheus	Time-series metrics	No native agent trajectory model	Excellent	No	SLOs, rates, histograms, infrastructure	Cannot explain an individual agent trajectory ⁵⁰
Sentry	Error/performance/agent tracing	Agent instrumentation available	Operational	Limited vs LangSmith	Correlating agent failures with application errors	Not primarily an agent-evaluation/dataset platform ⁵¹

The production pattern is therefore generally **composition**, not selection:

LangSmith	= semantic agent observability + evaluation
OpenTelemetry	= telemetry interoperability
Prometheus	= aggregate metrics/SLOs
Sentry	= application error/performance correlation

Long-running agents and durable execution

A long-running agent is not merely an agent with a large model timeout. It is an execution whose useful lifetime can exceed a single HTTP request, process lifetime, frontend connection, worker lease, or human response interval. It may pause for approval, wait for external systems, retry transient failures, continue after deployment/restart, or accumulate work over hours or days. LangGraph's checkpointer, interrupt, durability, replay, retry, timeout, and worker/runtime facilities are the primitives that make such execution practical. ⁵²

The key theoretical transition is:

WRONG MENTAL MODEL

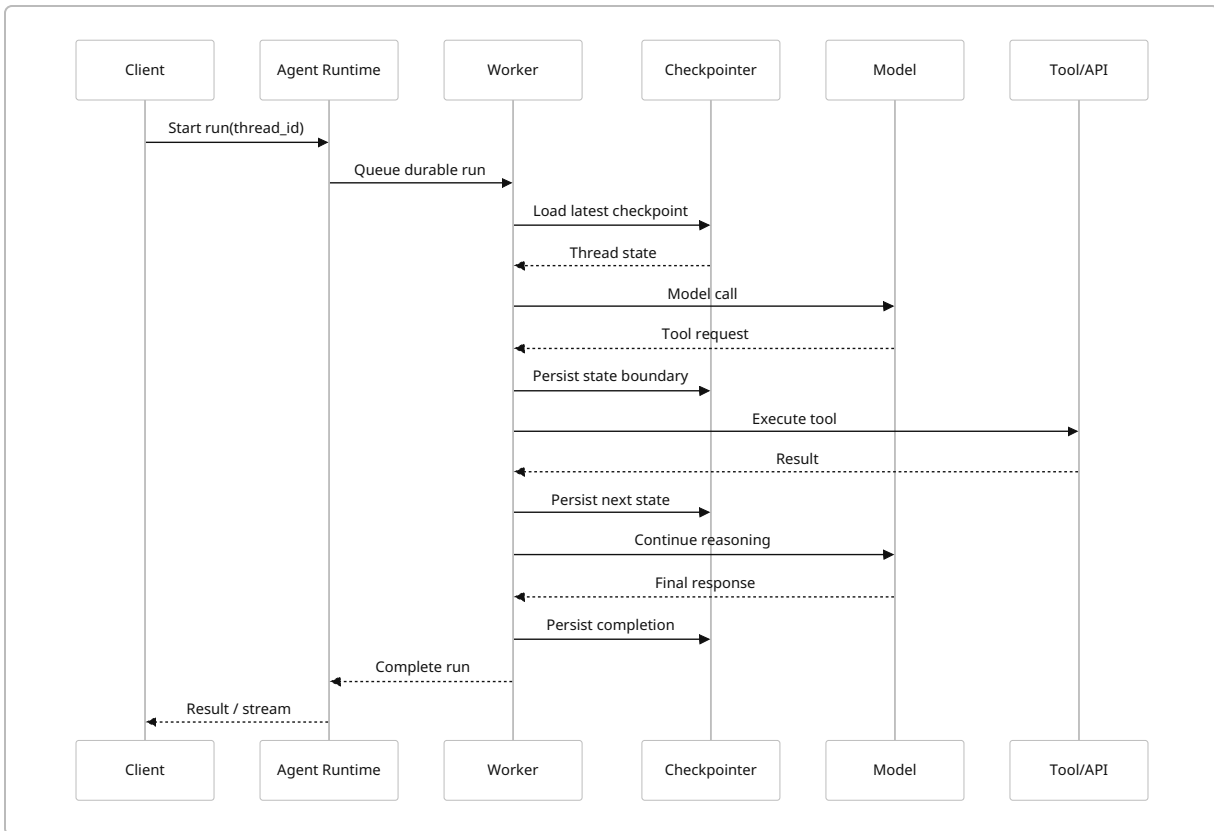
```
request
├─ giant Python function
│   └─ LLM loop
│       └─ hope nothing crashes
```

DURABLE MODEL

```
request
├─ persisted run
│   └─ node
│       └─ checkpoint
│           └─ node
│               └─ checkpoint
│                   └─ interrupt / retry / resume
```

LangGraph checkpoints at graph super-step boundaries rather than arbitrary lines inside a node. If execution later resumes after interruption or retry, the affected node executes again from the beginning. This is why node boundaries and side-effect design become part of your durability model. ⁵³

Checkpoint execution flow:



Agent Server implements a durable queue/worker architecture: API servers create pending runs, queue workers acquire and execute them, workers write checkpoints, and live output is broadcast through pub/sub. The queue enforces no more than one concurrently executing run for a given thread. Redis is used for ephemeral signaling/cancellation/streaming, while PostgreSQL stores run/thread and durable application data. ⁵⁴

Durability modes are one of the most important configuration surfaces. LangGraph exposes `exit`, `async`, and `sync` durability on graph execution methods. ⁵⁵

Mode	Write behavior	Crash window	Performance	Recommended use
<code>exit</code>	Persist only when graph exits, errors, or interrupts	Large during execution	Highest	Cheap/recomputable jobs without expensive intermediate effects
<code>async</code>	Persist in background while next step runs	Small	Strong default balance	General production agents
<code>sync</code>	Persist checkpoint before next step	Minimal at graph boundary	Highest checkpoint overhead	High-value workflows where replay is expensive/dangerous

The current LangGraph documentation describes `async` as the default background checkpointing behavior. `sync` provides the strongest state durability at graph boundaries, while `exit` cannot recover intermediate progress after a process-level failure. ⁵⁶

Usage is simple:

```
result = await graph.ainvoke(
    {"messages": [{"role": "user", "content": "Process the case"}]},
    config={
        "configurable": {
            "thread_id": "case-2026-8127",
        }
    },
    durability="sync",
)
```

A sensible policy is not necessarily to use `sync` everywhere. For example, a retrieval/planning node may be cheap to repeat under `async`, while a workflow involving external financial, provisioning, or destructive side effects should be structured so durable state is established immediately around the side-effect boundary. The exact mode should follow the business cost of replay, not a blanket “more durable is always better” rule. LangGraph’s modes explicitly trade persistence guarantees against execution overhead. ⁵⁵

Checkpoint backend strategy:

Checkpointner	Durability	Process restart	Operational complexity	Production suitability
In-memory	Low	No	Minimal	Tests/dev only
SQLite	Persistent local	Yes on same storage	Low	Local/single-instance development
PostgreSQL	Durable centralized	Yes	Moderate	General production
Agent Server managed persistence	Durable service-managed	Yes	Platform-managed	Strong fit when using LangSmith Deployment

LangGraph documents separate checkpoint packages including in-memory/development and persistent options such as SQLite and PostgreSQL, while Agent Server uses persistent platform storage for server resources and checkpoints. ⁵⁷

Deep Agents can directly inherit this durability by passing a checkpointner:

```

from deepagents import create_deep_agent
from langgraph.checkpoint.postgres import PostgresSaver

DB_URI = "postgresql://..."

with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    checkpointer.setup()

    agent = create_deep_agent(
        model="openai:gpt-5.5",
        tools=[...],
        checkpointer=checkpointer,
    )

    result = agent.invoke(
        {
            "messages": [
                {
                    "role": "user",
                    "content": "Investigate and prepare the remediation plan.",
                }
            ],
        },
        config={
            "configurable": {
                "thread_id": "incident-2026-08-31-42",
            }
        },
        durability="sync",
    )

```

The concrete database lifecycle should be managed outside request handlers in a real service rather than repeatedly creating database resources per request. The conceptual point is that Deep Agents accepts LangGraph persistence because its harness executes on LangGraph. ¹⁴

State size needs active design. By default, LangGraph checkpoints persist full channel values at each super-step. For long-lived, append-heavy threads this can create substantial storage growth. LangGraph 1.2 introduced `DeltaChannel` in beta to store incremental deltas for suitable state channels, trading storage efficiency against reconstruction behavior/API stability. ⁵⁵

This suggests separating:

```

Checkpoint
├─ current workflow status
├─ compact message state

```

- └─ IDs/references
- └─ pending decisions

Store

- └─ user memory
- └─ durable learned facts
- └─ cross-thread preferences

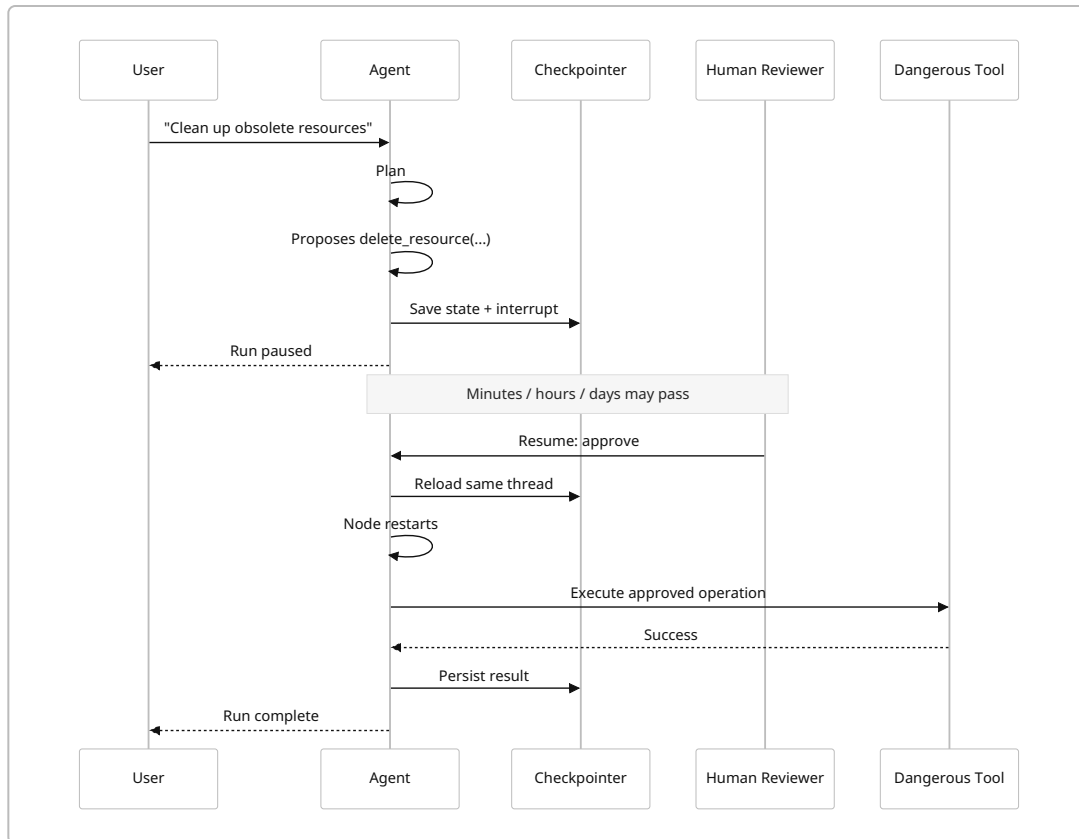
Object/artifact storage

- └─ large reports
- └─ datasets
- └─ images
- └─ generated archives
- └─ huge tool outputs

A common anti-pattern is placing entire large artifacts repeatedly into checkpoint state. Deep Agents already uses filesystem abstractions partly to externalize context and large tool results instead of endlessly stuffing everything into conversational messages. ⁵⁸

Human pauses are true durable workflow states. LangGraph's `interrupt()` persists the graph state through the configured checkpointer and can wait for external input. Resumption requires the same thread ID, and the value passed through `Command(resume=...)` becomes the return value of the interrupt. The node starts again from its beginning when resumed. ⁵⁹

Timeline:



Because code before the interrupt executes again, LangGraph explicitly recommends that side effects before an interrupt be idempotent or moved after the approval boundary/separated into another node. 60

A direct LangGraph interrupt pattern looks like:

```

from langgraph.types import interrupt, Command

def approval_node(state):
    approved = interrupt(
        {
            "kind": "approval",
            "operation": state["pending_operation"],
        }
    )

    if not approved:
        return {"status": "rejected"}

    return {"status": "approved"}
  
```

Initial run:

```

config = {
    "configurable": {
        "thread_id": "approval-case-481",
    }
}

result = graph.invoke(
    {"pending_operation": "delete customer export"},
    config=config,
)

```

Resume:

```

resumed = graph.invoke(
    Command(resume=True),
    config=config, # SAME thread_id
)

```

LangGraph requires interrupt payloads/resume values to be serializable and the same persisted thread to be used when continuing. ⁵⁹

Deep Agents provides an even higher-level tool-approval interface:

```

from deepagents import create_deep_agent
from langgraph.checkpoint.memory import MemorySaver

agent = create_deep_agent(
    model="openai:gpt-5.5",
    tools=[
        read_customer,
        update_customer,
        delete_customer,
    ],
    checkpointer=MemorySaver(),
    interrupt_on={
        "read_customer": False,
        "update_customer": {
            "allowed_decisions": ["approve", "edit", "reject"]
        },
        "delete_customer": {
            "allowed_decisions": ["approve", "reject"]
        },
    },
)

```



```
    },
)
```

Deep Agents documents `interrupt_on` for tool-level approval, and LangChain's lower-level `HumanInTheLoopMiddleware` provides equivalent middleware-based configuration for `create_agent`. Both rely on a checkpointer for durable interruption. ⁶¹

Retries must distinguish transient from deterministic failure. LangGraph's `RetryPolicy` defaults are currently:

Parameter	Default
<code>max_attempts</code>	3, including initial attempt
<code>initial_interval</code>	0.5 seconds
<code>backoff_factor</code>	2.0
<code>max_interval</code>	128.0 seconds
<code>jitter</code>	True
<code>retry_on</code>	<code>default_retry_on</code>

The default retry predicate excludes common programming/logic errors such as `ValueError`, `TypeError`, `RuntimeError`, syntax/name/import errors, and `OSError`; for common HTTP clients it retries 5xx responses, and `NodeTimeoutError` is retryable by default. ¹⁵

Example:

```
from langgraph.types import RetryPolicy

builder.add_node(
    "external_research",
    external_research,
    retry_policy=RetryPolicy(
        max_attempts=4,
        initial_interval=1.0,
        backoff_factor=2.0,
        max_interval=30.0,
        jitter=True,
        retry_on=(ConnectionError, TimeoutError),
    ),
)
```

Retries should be used for transient faults, not as a generic response to every exception. Otherwise a programming bug or authorization failure can be multiplied into repeated cost and repeated side-effect

attempts. LangGraph's default predicate deliberately avoids retrying many deterministic programming errors. ¹⁵

Timeouts are per-attempt controls, not job lifetimes. LangGraph 1.2 supports node timeout policies with a hard `run_timeout` and/or an `idle_timeout` that resets when progress is observed. Node timeouts apply to async nodes; synchronous nodes with node timeout configuration are rejected at compile time. An idle-timed long operation can explicitly call `runtime.heartbeat()`. ⁶²

```
from langgraph.types import TimeoutPolicy

async def long_analysis(state, runtime):
    for batch in get_batches():
        await process_batch(batch)
        runtime.heartbeat()

    return {"analysis_complete": True}

builder.add_node(
    "long_analysis",
    long_analysis,
    timeout=TimeoutPolicy(
        run_timeout=60 * 30,
        idle_timeout=60,
        refresh_on="heartbeat",
    ),
)
```

If a timeout occurs, LangGraph raises `NodeTimeoutError`, clears the failed attempt's buffered writes, and lets the retry policy decide whether another attempt should be made. ⁶³

Error handling occurs after retries. LangGraph 1.2 node error handlers receive `NodeError` context and can return `Command` to update state and route execution to a recovery/compensation node. This makes saga-like compensation patterns possible inside the graph. ⁶⁴

```
from langgraph.errors import NodeError
from langgraph.types import Command, RetryPolicy

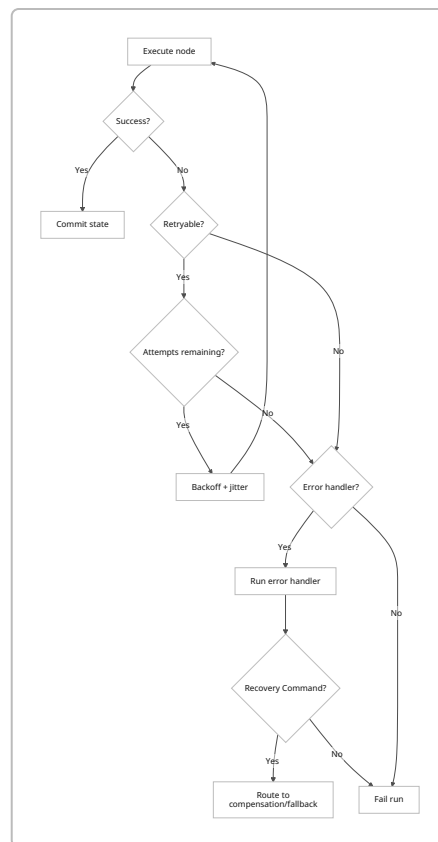
def compensate_reservation(state, error: NodeError) -> Command:
    return Command(
        update={
            "status": "compensation_required",
            "failure": type(error.error).__name__,
        },
        goto="release_resources",
    )
```

```

builder.add_node(
    "reserve_resource",
    reserve_resource,
    retry_policy=RetryPolicy(
        max_attempts=3,
        retry_on=ConnectionError,
    ),
    error_handler=compensate_reservation,
)

```

The runtime order is:



This ordering—attempt → timeout/exception → retry policy → exhausted retries → error handler—is explicitly defined by LangGraph’s fault-tolerance runtime. 65

Exactly-once side effects are not magically provided by checkpointing. Consider:

1. Tool charges credit card.
2. Process crashes.
3. Checkpoint recording the success was not committed.

4. Runtime resumes.
5. Tool call executes again.

The durable graph cannot know whether an external non-transactional side effect committed unless the application makes it discoverable. LangGraph therefore emphasizes idempotency for repeatable nodes. 6

Use one of these patterns:

Pattern	Mechanics	Strength	Weakness
Idempotency key	Stable operation ID sent to external API	Excellent where supported	External API must honor it
Upsert	Repeated write converges	Simple	Not suitable for every side effect
Read-before-write	Query state before changing	Broad applicability	Race conditions without locking
Transaction + outbox	Record intent/event atomically	Strong distributed pattern	More infrastructure
Saga compensation	Undo already-completed steps	Works across services	Compensation may itself fail
Human approval	Pause before irreversible effect	Strong governance	Human latency
Deduplication ledger	Record external-effect operation IDs	General	Ledger must itself be durable

For a harness intended to “build whatever you want,” **idempotency should be a tool-interface requirement**, not something every agent developer remembers individually.

For example:

```
from dataclasses import dataclass

@dataclass(frozen=True)
class ToolContext:
    thread_id: str
    run_id: str
    operation_id: str

async def transfer_funds(
    *,
    account_from: str,
    account_to: str,
```

```

    amount: float,
    ctx: ToolContext,
):
    # Provider must guarantee repeated use of operation_id
    # returns the original transaction instead of issuing another transfer.
    return await payment_api.transfer(
        source=account_from,
        destination=account_to,
        amount=amount,
        idempotency_key=ctx.operation_id,
    )

```

Concurrency and “double texting.” Agent Server defaults to enqueueing additional runs on an already-active thread, executing them sequentially. Other supported strategies include rejecting the second run, interrupting the existing run, and rollback-oriented behavior. Interrupting mid-tool requires careful handling because an external tool may already have begun an operation even though the graph has been interrupted. ⁶⁶

For most business workflow agents:

```

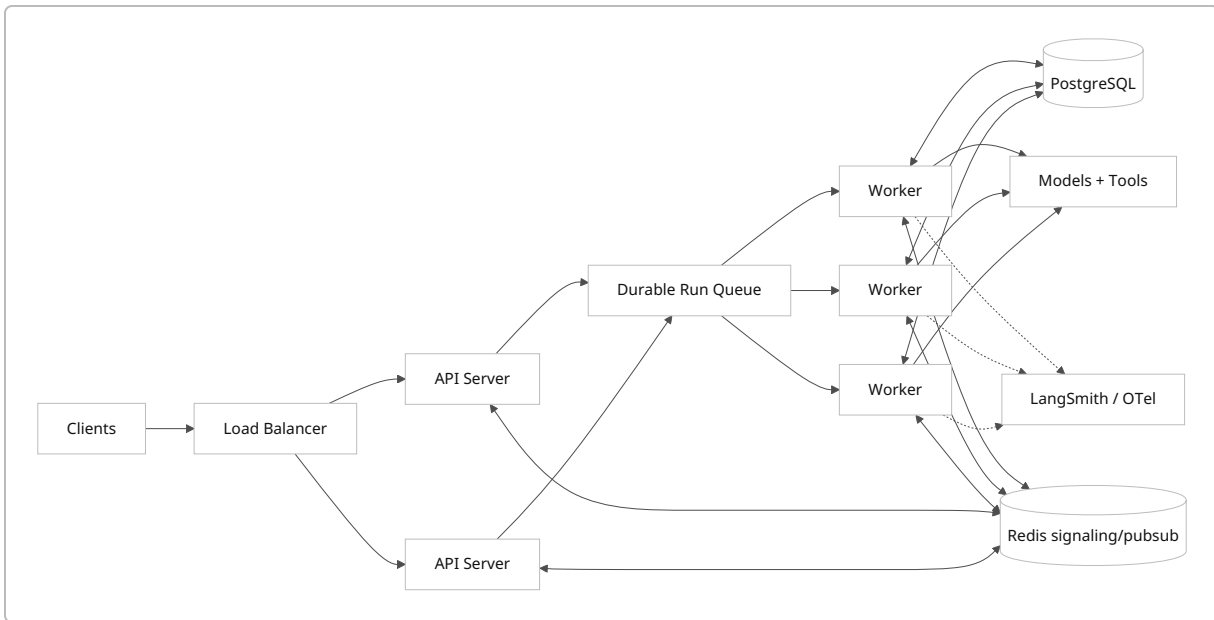
same thread + current run active
├── normal chat          → enqueue
├── exclusive workflow → reject
├── interactive replanning → interrupt carefully
└── speculative work    → separate thread

```

This avoids racing two copies of the same durable state machine.

Worker architecture matters for long-running execution. In Agent Server, API concurrency and run concurrency are separate. Queue workers execute up to `N_JOBS_PER_WORKER` runs concurrently; the documented default is 10. Increasing this value improves concurrency only when worker CPU, memory, network, model/tool workloads, and database capacity can support it. ⁶⁷

A production topology is therefore:



Agent Server supports single-host, split API/queue-worker, and distributed runtime configurations, allowing API request handling and graph execution to be scaled independently. ⁶⁸

Long-running observability should add runtime-specific metrics:

```

queued_runs
queue_wait_seconds
running_runs
run_age_seconds
node_attempts
retry_count
checkpoint_write_latency
checkpoint_write_failures
interrupt_age_seconds
stale_interrupts
worker_lease_age
worker_utilization
resume_count
compensation_count
cancel_count
orphaned_external_operation_count

```

LangSmith tracks execution-level latency/error/tool/model/cost signals, while queue/worker/checkpoint metrics are better exported through the runtime's infrastructure telemetry stack and OpenTelemetry/Prometheus. ⁶⁹

Long-running evaluation must evaluate process quality. Besides answer quality, test:

Can a job resume after worker death?
Does replay repeat irreversible side effects?
Can every HITL pause resume after hours?
What happens when the model API fails on attempt 2?
What happens when the checkpoint database fails?
What happens when a tool returns after cancellation?
Does graph version N+1 resume a thread started on N?
Does an outdated approval authorize a changed action?
Do retries respect maximum cost?
Does a 24-hour thread exceed state/storage budgets?

LangGraph explicitly notes that resumed threads execute the graph code that is compiled at the time of resume, so application versioning and state-schema compatibility need to be treated as deployment concerns. ⁶

The best durability test is therefore **fault injection**, not a happy-path unit test:

Inject:
process SIGTERM
worker loss
network timeout
model 429/5xx
database connection reset
slow tool
duplicate webhook
malformed tool result
repeated resume
stale approval
concurrent new input

Then verify the invariant:

recovery must be
safe + deterministic enough + bounded + observable

Agent security and threat models

Agent security differs from conventional chatbot moderation because a harness connects untrusted natural-language inputs and probabilistic model decisions to **capabilities**: files, databases, shells, APIs, credentials, browsers, memory, subagents, MCP servers, and external business systems. NIST has highlighted that agentic systems inherit conventional software risks while also introducing risks resulting from model outputs being connected to consequential actions; OWASP's GenAI guidance identifies prompt injection and excessive agency among major LLM/agent risks. ⁷⁰

The right security model is:

LLM output = untrusted proposal

NOT

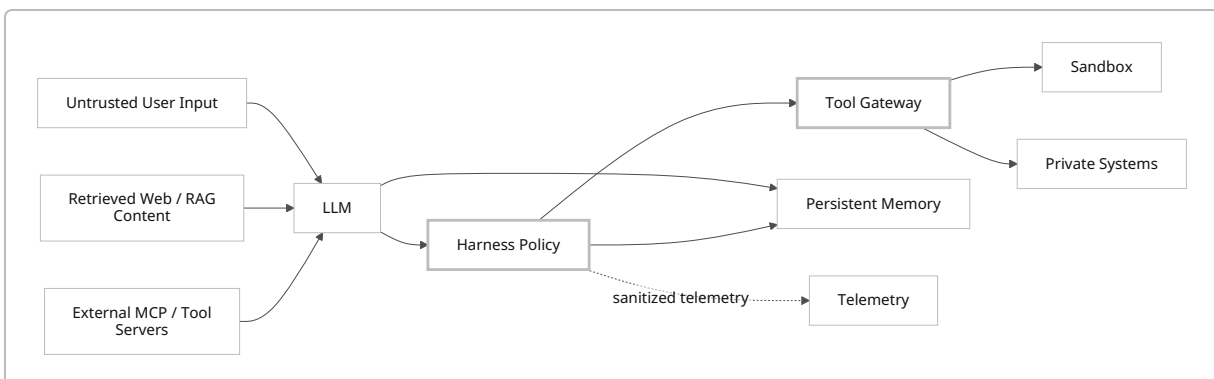
LLM output = authorization

Even a system prompt that says “never delete production data” is not a sufficient authorization system. Deep Agents’ own RAG documentation warns that retrieved content can contain indirect prompt injections and that prompt/delimiter strategies do not reliably eliminate the risk. ⁷¹

Assets to protect:

Secrets / credentials
PII and confidential documents
Filesystem and source code
Databases
Money / financial operations
External SaaS accounts
Cloud / infrastructure resources
Agent persistent memory
Organization-wide policy memory
Tool permissions
Other users' threads
Artifacts
Telemetry
Evaluation datasets
Human approvals

Trust boundaries:



The important design decision is that **the harness and tool gateway are trust boundaries; the model is not.**

Threat model

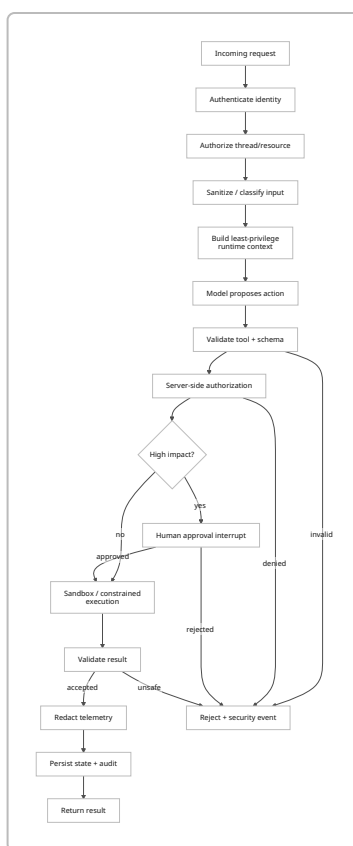
Threat	Attack path	Potential impact	Primary harness control
Direct prompt injection	Malicious user instructions	Tool abuse, policy bypass	Authorization outside prompt; tool allowlists; HITL
Indirect prompt injection	Web page, RAG document, email, tool result	Model follows attacker instructions	Treat retrieved content as untrusted; capability isolation; policy enforcement
Excessive agency	Model has unnecessary high-impact tools	Accidental/destructive actions	Least privilege, approvals, narrower tools
Secret exfiltration	Agent sees secret + network capability	Credential theft	Secret isolation; sandbox network policy; redaction
Memory poisoning	Attacker writes shared memory	Future conversations compromised	Per-user namespaces; read-only shared memory
Cross-tenant access	Broken namespace/authz	Confidentiality breach	AuthN/AuthZ + tenant-scoped stores/checkpoints
Shell/code abuse	Model executes commands	Host compromise	Sandbox, resource limits, no host shell
Tool schema abuse	Manipulated arguments	Unauthorized mutations	Server-side validation and authorization
MCP/tool supply chain	Malicious/compromised server	Data theft, malicious outputs/actions	Server allowlists, scopes, isolation, auditing
Cost exhaustion	Prompt induces loops/subagents/tools	Financial/availability impact	Call limits, recursion/budget policy, rate limits
Telemetry leakage	Prompts/results logged verbatim	PII/secrets stored in observability backend	Trace masking/redaction/conditional tracing
Unsafe retry	Destructive effect repeated	Duplicate payment/delete/etc.	Idempotency and compensation
Approval confusion	Human approves misleading/stale summary	Wrong side effect	Present exact normalized operation and arguments
Subagent escalation	Delegated agent has broader capabilities	Privilege amplification	Capability-scoped subagents

Threat	Attack path	Potential impact	Primary harness control
Denial of service	Heavy shell/tool/model use	CPU/memory/API exhaustion	sandbox quotas, timeouts, concurrency limits

These attack classes are consistent with OWASP's prompt-injection/excessive-agency guidance, NIST's work on agent-system security, current prompt-injection research, and Deep Agents' explicit warnings around RAG, shared memory, filesystem, and shell access. ⁷²

Recent research strengthens the argument for runtime controls: prompt-injection and agent-hijacking studies show that instruction-level defenses alone are brittle, especially when agents consume adversarial external content and possess tools. System-level isolation and policy enforcement are therefore required even when prompts instruct the model to ignore malicious content. ⁷³

Defense-in-depth execution flow:



This matters because filtering only at input misses indirect prompt injection; filtering only at output misses harmful side effects that happened before the final answer. Security controls have to sit **around each capability boundary**. ⁷⁴

Authentication and authorization. Agent Server supports custom authentication using `@auth.authenticate`; the returned identity enters the runtime context. Authorization handlers

registered with `@auth.on` can filter or deny access to threads, runs, assistants, cron resources, and other platform objects. ⁷⁵

Conceptually:

```
from langgraph_sdk import Auth

auth = Auth()

@auth.authenticate
async def authenticate(authorization: str | None):
    user = await validate_bearer_token(authorization)
    return {
        "identity": user.id,
        "tenant_id": user.tenant_id,
        "roles": user.roles,
    }

@auth.on
async def authorize(ctx, value):
    tenant_id = ctx.user["tenant_id"]

    # Resources created by this caller become tenant scoped,
    # and reads/searches are restricted to the same tenant.
    if isinstance(value, dict):
        value.setdefault("metadata", {})[tenant_id] = tenant_id

    return {
        "tenant_id": tenant_id,
    }
```

The exact authorization logic should normally be **deny by default** in application policy and should never rely on the model to decide whether a caller may access a resource. Agent Server's custom-auth system gives authorization handlers access to authenticated user context for this purpose. ⁷⁵

The tool layer must reauthorize. A user being allowed to access an agent does not imply that the user may perform every action the agent can theoretically call. Tool functions should receive validated user/tenant identity and enforce business authorization again:

```
async def delete_invoice(
    invoice_id: str,
    *,
    runtime,
) -> str:
    user = runtime.context.user
```

```

invoice = await invoices.get(invoice_id)

if invoice.tenant_id != user.tenant_id:
    raise PermissionError("Cross-tenant access denied")

if "billing-admin" not in user.roles:
    raise PermissionError("Deletion requires billing-admin")

await invoices.delete(invoice_id)
return "Deleted"

```

This protects against a confused-deputy scenario where the model is manipulated into requesting a tool call beyond the caller’s actual authority.

Deep Agents filesystem security deserves special attention. Deep Agents exposes filesystem-like capabilities through pluggable backends. The default `StateBackend` stores files in graph state and keeps them thread-scoped, whereas `FilesystemBackend` accesses actual disk and `LocalShellBackend` adds host shell execution. ⁷⁶

Risk ranking:

Backend	Host filesystem	Shell	Isolation	Recommended production role
<code>StateBackend</code>	No direct host access	No	Logical graph-state boundary	Safe default scratch workspace
<code>StoreBackend</code>	Store abstraction	No	Namespace/policy dependent	Persistent controlled memory
<code>FilesystemBackend</code>	Yes	No	Depends on configuration	Controlled environments only
<code>LocalShellBackend</code>	Yes	Yes	None	Development only
Sandbox backend	Isolated filesystem	Usually sandboxed execute	Provider/container boundary	Production code execution

Deep Agents explicitly warns that `FilesystemBackend` can read accessible secrets and permanently modify files. For path containment, `virtual_mode=True` should be used with a root directory; the default `virtual_mode=False` does **not** make `root_dir` a security boundary. ⁷⁷

Safer filesystem pattern:

```

from deepagents import create_deep_agent
from deepagents.backends import (
    CompositeBackend,
    StateBackend,
    FilesystemBackend,
)

backend = CompositeBackend(
    default=StateBackend(),
    routes={
        "/workspace/": FilesystemBackend(
            root_dir="/srv/agent-workspace",
            virtual_mode=True,
        ),
    },
)

agent = create_deep_agent(
    model="openai:gpt-5.5",
    backend=backend,
)

```

This leaves internal/scratch agent files in thread-scoped state and exposes only the explicitly routed workspace to disk. Deep Agents documents this `CompositeBackend` pattern as a way to separate internal state from a controlled project filesystem. ⁷⁸

`LocalShellBackend` is much more dangerous. It runs commands directly on the host via `subprocess.run(shell=True)` with no sandboxing; its documented defaults include a 120-second command timeout and `max_output_bytes=100000`, but those limits do not turn host shell execution into a security boundary. The documentation explicitly warns against production, multi-tenant, or untrusted-input use. ⁷⁹

Therefore:

```

# Fine for a controlled developer workstation.
from deepagents.backends import LocalShellBackend

backend = LocalShellBackend(
    root_dir=".",
    virtual_mode=True,
    env={"PATH": "/usr/bin:/bin"},
)

```

should **not** be interpreted as:

```
virtual_mode=True
=
secure shell sandbox
```

Deep Agents states the opposite: shell commands can still access other host paths; `virtual_mode` does not secure unrestricted shell execution. ⁸⁰

For production execution, use a sandbox backend with isolation and then independently constrain filesystem, network, credentials, CPU, memory, runtime, process count, and disk. Deep Agents supports sandbox backends and frames them as isolated environments for file manipulation and command execution. ⁸¹

A production sandbox policy should look conceptually like:

```
sandbox:
  filesystem:
    writable:
      - /workspace
      - /tmp
    read_only:
      - /reference
  network:
    default: deny
    allow:
      - approved-api.internal
  resources:
    cpu: bounded
    memory: bounded
    disk: bounded
    processes: bounded
    execution_time: bounded
  credentials:
    inject_only_when_needed: true
    scope: per_tool_or_request
  lifecycle:
    isolation: per_thread_or_job
    cleanup_on_completion: true
```

These are architectural recommendations rather than a claim that a single Deep Agents configuration object exposes all of those fields; the exact quotas/network implementation depends on the chosen sandbox provider. Deep Agents' core guidance is to use isolated sandboxes instead of exposing host execution. ⁸²

Filesystem permissions are a separate control from the backend. Deep Agents supports `FilesystemPermission` rules that are evaluated before built-in filesystem operations reach the backend,

and backend wrapping/subclassing can add policy hooks for content inspection, audit logging, rate limits, or custom enterprise rules. ⁸³

Example:

```
from deepagents import (
    create_deep_agent,
    FilesystemPermission,
)

agent = create_deep_agent(
    model="openai:gpt-5.5",
    backend=backend,
    permissions=[
        FilesystemPermission(
            operations=["write"],
            paths=["/policies/**"],
            mode="deny",
        ),
    ],
)
```

A critical distinction is that **filesystem permissions should not be assumed to govern arbitrary shell commands**. Shell execution requires enforcement at the sandbox/backend/policy layer because a shell can otherwise directly access paths independently of the high-level `read_file` / `write_file` tools. Deep Agents distinguishes the built-in filesystem policy surface from `execute`-capable backends. ⁸⁴

For custom policy logic:

```
from deepagents.backends.filesystem import FilesystemBackend
from deepagents.backends.protocol import WriteResult

class GuardedBackend(FilesystemBackend):
    def write(self, file_path: str, content: str) -> WriteResult:
        if file_path.startswith("/policies/"):
            return WriteResult(
                error="Agent writes to organization policies are forbidden"
            )

        if contains_secret(content):
            return WriteResult(
                error="Potential secret material rejected"
            )
```

```
audit_write(file_path, content)
return super().write(file_path, content)
```

Deep Agents explicitly documents backend subclassing/wrapping for validation beyond declarative path permissions. ⁸³

Persistent memory creates a novel security boundary. A writable memory entry is effectively future context. If attacker-controlled content survives into later runs, it can behave like a stored prompt injection. This becomes even worse if multiple users consume the same writable memory namespace. Deep Agents explicitly warns that shared memory is a prompt-injection vector and recommends user scoping by default plus read-only access for shared organization policies. ¹¹

Safe architecture:

```
/user/{user_id}/memory/*
  read/write by that user's agent

/org/{org_id}/policies/*
  read by agents
  written only by trusted application/admin workflow

/assistant/{assistant_id}/learned/*
  writable only if explicit product requirement
  reviewed/sanitized before cross-user exposure
```

Deep Agents recommends per-user namespaces for most application memories and explicitly describes organization memory as typically read-only. ¹¹

A production memory store should therefore persist provenance:

```
{
  "memory_id": "mem_8127",
  "tenant_id": "tenant-a",
  "scope": "user",
  "source_thread_id": "thread-481",
  "source_run_id": "run-993",
  "created_by": "agent",
  "approved": true,
  "trust_level": "user-derived",
  "content": "...
}
```

The model should be informed that memory has provenance/trust levels, but enforcement should happen in retrieval/permissions outside the model.

Indirect prompt injection requires capability isolation. LangChain's Deep Agents RAG documentation explicitly states that retrieved documents can contain instructions masquerading as data and that prompt/delimiter defenses are not reliable protection. Therefore the robust strategy is not merely:

"Ignore instructions in retrieved documents."

It is:

```
untrusted content
  ↓
model may be influenced
  ↓
but model possesses only narrowly-scoped capabilities
  ↓
tool gateway performs independent authorization
  ↓
high-impact operations require approval
```

85

Human-in-the-loop is a security primitive, but only when designed correctly. LangChain's `HumanInTheLoopMiddleware` can require approval, editing, or rejection of selected tools and requires a checkpoint so pauses survive between requests. 86

```
from langchain.agents.middleware import HumanInTheLoopMiddleware

hitl = HumanInTheLoopMiddleware(
    interrupt_on={
        "search_public_docs": False,
        "send_email": {
            "allowed_decisions": [
                "approve",
                "edit",
                "reject",
            ]
        },
        "delete_database_record": {
            "allowed_decisions": [
                "approve",
                "reject",
            ]
        },
    },
)
```

```
}  
)
```

However, approval UI is itself part of the security boundary. Do not ask:

```
"Agent wants to perform an action. Approve?"
```

Ask:

```
Action: delete_database_record  
Tenant: ACME  
Target: customer_record_981  
Reason: duplicate record  
Arguments:  
    record_id = 981  
    hard_delete = true  
  
This operation is irreversible.  
  
[Reject] [Approve]
```

The approval should be cryptographically/logically bound to the exact normalized operation parameters. If the arguments change after approval, require another approval. This is a production security pattern derived from the fact that LangGraph resumes workflow state after an interrupt rather than turning approval into universal future authority. ⁸⁷

PII middleware protects the agent data path. LangChain's `PIIMiddleware` can detect configured PII classes and redact/mask them. Current documentation notes that with `apply_to_output=True`, recent LangChain versions can also transform streamed output, including text deltas/tool-related output/state snapshots. ⁸⁸

```
from langchain.agents.middleware import PIIMiddleware  
  
pii = [  
    PIIMiddleware(  
        "email",  
        strategy="redact",  
        apply_to_input=True,  
        apply_to_output=True,  
    ),  
    PIIMiddleware(  
        "credit_card",  
        strategy="mask",  
        apply_to_input=True,  
    ),  
]
```

```

        apply_to_output=True,
    ),
]

```

But there are **two different PII problems**:

```

Agent PII control
  PIIMiddleware
    ↓
what reaches/leaves model/tool workflow

Telemetry PII control
  LangSmith redaction / OTel collector
    ↓
what reaches logs/traces/APM

```

You often need both. Redacting the final response does not retroactively remove a secret already emitted into a trace, and trace masking does not stop the agent itself from sending sensitive data to an unauthorized tool. ⁸⁹

Encryption at rest. Agent Server supports AES-based encryption of checkpoint blobs and selected JSON fields. Current documentation accepts AES key lengths of 16, 24, or 32 bytes and supports `LANGGRAPH_AES_JSON_KEYS` for selected fields across thread/run/assistant/cron/store data; encrypted fields become unavailable for search/filtering. ⁹⁰

Example configuration:

```

export LANGGRAPH_AES_KEY="..."
export LANGGRAPH_AES_JSON_KEYS="api_key,secret_token,user_credentials"

```

Encryption is important, but it does not solve authorization, prompt injection, telemetry exposure, or over-privileged tools.

Security observability needs its own signals. Record events such as:

```

authentication_failure
authorization_denial
cross_tenant_access_attempt
tool_permission_denial
sandbox_policy_denial
unexpected_network_destination
secret_detection
pii_redaction

```

```
human_approval_requested
human_approval_rejected
human_approval_edited
tool_limit_exceeded
model_limit_exceeded
memory_write_denied
shared_memory_write_attempt
suspicious_tool_sequence
prompt_injection_detector_score
```

These can be recorded as LangSmith metadata/feedback where appropriate, OpenTelemetry events/spans for cross-service correlation, Prometheus counters for rates, and Sentry events for security-relevant exceptions. The systems serve complementary roles rather than replacing one another. ⁹¹

Security evaluation should be adversarial. Build datasets containing:

```
direct prompt injection
indirect web-page injection
malicious RAG documents
malicious email content
memory poisoning
cross-tenant requests
tool-argument injection
SSRF attempts
path traversal attempts
secret exfiltration requests
resource-exhaustion prompts
approval bypass attempts
subagent privilege escalation
malformed tool responses
fake "system messages" inside tool output
```

NIST's agentic-security work and recent agent-hijacking research make clear that realistic evaluation needs to test complete tool-using trajectories under adversarial input rather than only whether the final textual answer appears safe. ⁹²

For every attack case, evaluate at least:

```
Did forbidden side effect occur?           ← most important
Did unauthorized data leave trust boundary?
Did agent attempt forbidden action?
Did harness block it?
Was block observable?
Did agent recover safely afterward?
```

Did retries or delegation bypass control?
Did telemetry leak the sensitive payload?

A final answer such as “I cannot do that” should still fail the test if the agent first exfiltrated a secret and only then produced a safe-looking response.

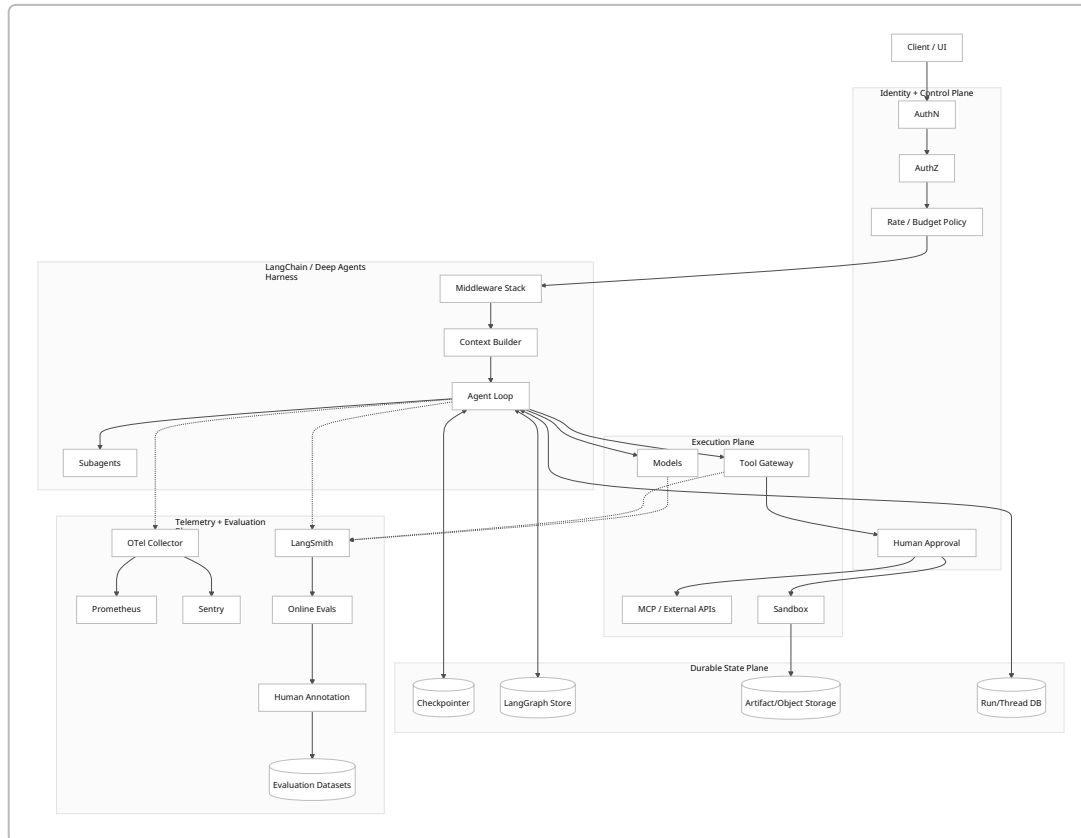
Security controls: pros and cons

Control	Advantages	Weaknesses
System prompt policy	Cheap, useful steering	Not a security boundary; vulnerable to injection
Tool allowlist	Strong capability reduction	Coarse if tools themselves are broad
Filesystem permissions	Deterministic path control	Does not automatically constrain arbitrary shell
HITL	Strong for consequential actions	Latency, fatigue, approval UX risk
Sandbox	Limits blast radius	Needs network/secret/resource policy too
AuthN/AuthZ	Strong identity/resource control	Must also exist inside tools
Per-user memory	Strong tenant isolation	More storage/management
Read-only shared memory	Prevents persistent policy poisoning	Reduces autonomous learning
PIIMiddleware	Protects agent data path	Detection can be incomplete
Trace redaction	Protects observability data	Can reduce debugging context
Model/tool call limits	Bounds runaway behavior	Does not understand business impact
Retries	Improves transient reliability	Can amplify harmful non-idempotent effects
Encryption	Protects stored data	Does not mitigate runtime misuse
LLM safety judge	Flexible detection	Probabilistic; not suitable as sole enforcement

The recurring pattern is that deterministic controls should enforce **authority and capability**, while model-based controls can help classify, prioritize, or detect suspicious semantic behavior.

Integrated production architecture and operating patterns

Putting the three topics together produces a harness architecture with distinct **decision**, **control**, **execution**, **state**, and **telemetry** planes.



This architecture maps cleanly onto LangChain/Deep Agents and LangGraph primitives: middleware handles cross-cutting agent behavior, Deep Agents provides higher-level workspace/subagent capabilities, LangGraph supplies persistent execution, Agent Server can supply worker/queue infrastructure, and LangSmith provides semantic trace/evaluation facilities. ⁹³

A strong production middleware composition might look conceptually like:

```
from langchain.agents.middleware import (
    PIIMiddleware,
    HumanInTheLoopMiddleware,
    ModelCallLimitMiddleware,
    ToolCallLimitMiddleware,
    ToolRetryMiddleware,
    ModelRetryMiddleware,
)

middleware = [
```

```

# Data protection
PIIMiddleware(
    "email",
    strategy="redact",
    apply_to_input=True,
    apply_to_output=True,
),

# Model-level resilience
ModelRetryMiddleware(...),

# Tool-level resilience
ToolRetryMiddleware(...),

# Budget boundaries
ModelCallLimitMiddleware(
    run_limit=12,
    thread_limit=100,
    exit_behavior="end",
),

ToolCallLimitMiddleware(
    run_limit=20,
    thread_limit=150,
    exit_behavior="error",
),

# Consequential action control
HumanInTheLoopMiddleware(
    interrupt_on={
        "read_data": False,
        "send_email": {
            "allowed_decisions": [
                "approve",
                "edit",
                "reject",
            ]
        },
        "delete_data": {
            "allowed_decisions": [
                "approve",
                "reject",
            ]
        },
    },
),
]

```

LangChain ships prebuilt middleware for tool/model retry, fallback, summarization, HITL, model/tool-call limits, PII handling, tool selection, filesystem, subagents, and other cross-cutting capabilities. The exact order needs to be designed because middleware composition determines what data each layer sees and whether an exception is retried, sanitized, interrupted, or surfaced. ⁸⁶

For Deep Agents, much of the harness can be composed directly through `create_deep_agent`:

```
from deepagents import create_deep_agent, FilesystemPermission
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend

agent = create_deep_agent(
    model="openai:gpt-5.5",

    tools=[
        search_docs,
        read_customer,
        update_customer,
        send_email,
    ],

    backend=CompositeBackend(
        default=StateBackend(),
        routes={
            "/memories/": StoreBackend(
                namespace=lambda rt: (
                    rt.server_info.user.identity,
                )
            ),
        },
    ),

    permissions=[
        FilesystemPermission(
            operations=["write"],
            paths=["/policies/**"],
            mode="deny",
        ),
    ],

    interrupt_on={
        "read_customer": False,
        "update_customer": {
            "allowed_decisions": ["approve", "edit", "reject"]
        },
        "send_email": {
            "allowed_decisions": ["approve", "edit", "reject"]
        },
    },
```



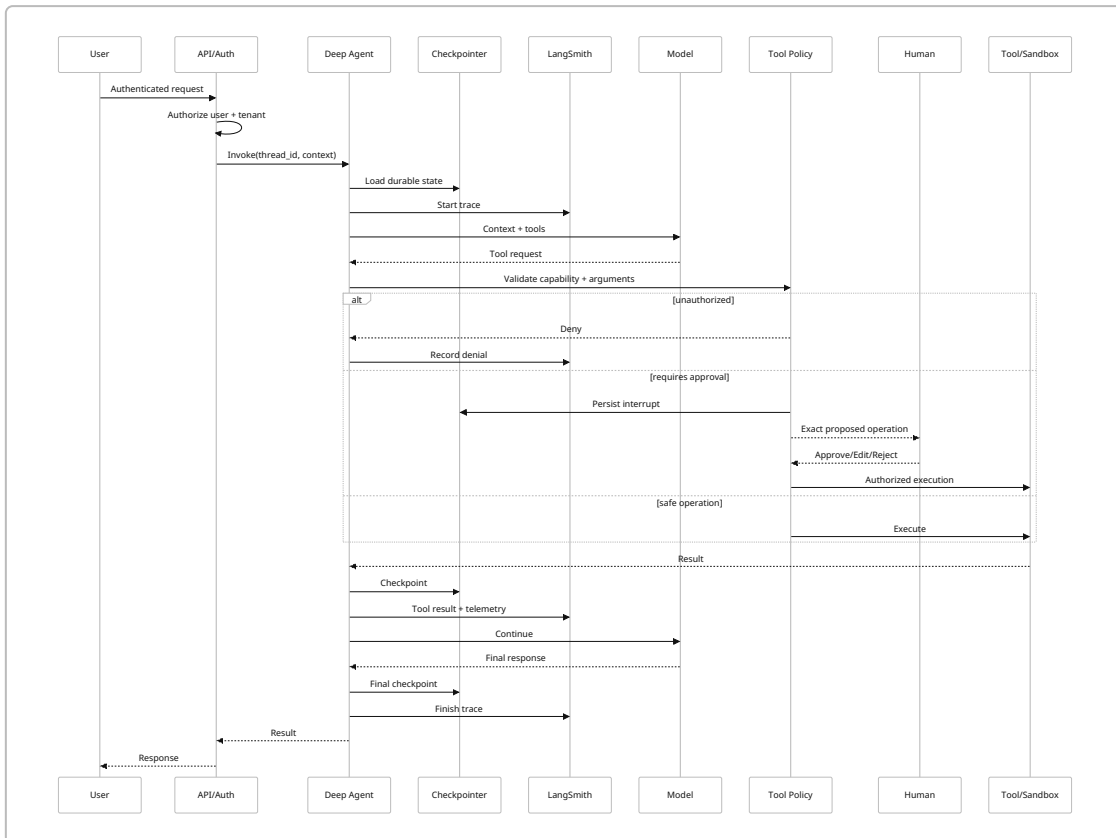
```

    },
    checkpointer=production_checkpointer,
)

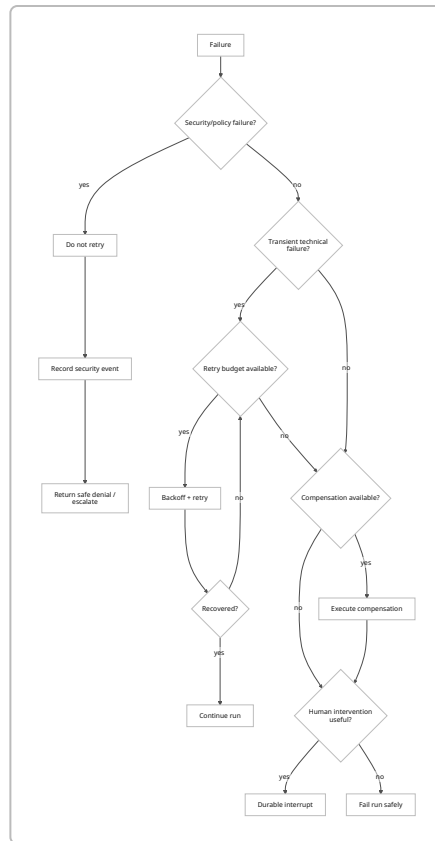
```

Deep Agents' default backend is `StateBackend`; persistent cross-thread memory can be introduced explicitly with a Store-backed route, and human-in-the-loop requires checkpointing. 94

The **production request flow** should then be:



The **failure flow** should be explicit enough that on-call engineers can predict what happens without reading agent prompts:



This is a crucial distinction: **an authorization denial is not a transient error**. Retrying it three times is not resilience. Likewise, a content-policy rejection, invalid schema, or cross-tenant access attempt should generally leave the retry path immediately. LangGraph lets developers control `retry_on` precisely for this reason. ¹⁵

A recommended failure taxonomy is:

Failure class	Retry?	Persist?	Alert?	Recovery
Provider 429/5xx	Usually	Yes	On sustained rate	Backoff/fallback
Network timeout	Usually	Yes	On threshold	Retry
Invalid model output	Limited	Yes	Metric	Structured retry/repair
Tool validation error	Usually no	Yes	Metric	Return tool error/model replan
AuthZ denial	No	Yes/audit	Security metric	Deny
Prompt-injection suspicion	No automatic privileged retry	Yes safely	Security	Restrict/escalate

Failure class	Retry?	Persist?	Alert?	Recovery
Checkpoint backend failure	Carefully	Yes when possible	High severity	Halt/drain/recover DB
Worker death	Runtime restart/resume	Checkpoint is source	Infra alert	Resume
Irreversible tool uncertain state	Never blind retry	Yes	High severity	Reconcile by idempotency key
Human rejection	No	Yes	Normal audit	Replan/end
Budget exhausted	No	Yes	Cost metric	Graceful stop/escalation
Sandbox policy violation	No	Audit	Security	Stop capability

Observability storage and application state should use different retention policies.

A useful production policy is:

```
Checkpoint state:
  retain while thread/job is live
  + operational recovery window

Persistent memory:
  retain according to application/business purpose

Audit/security events:
  retain according to compliance/security policy

Full agent traces:
  short retention by default

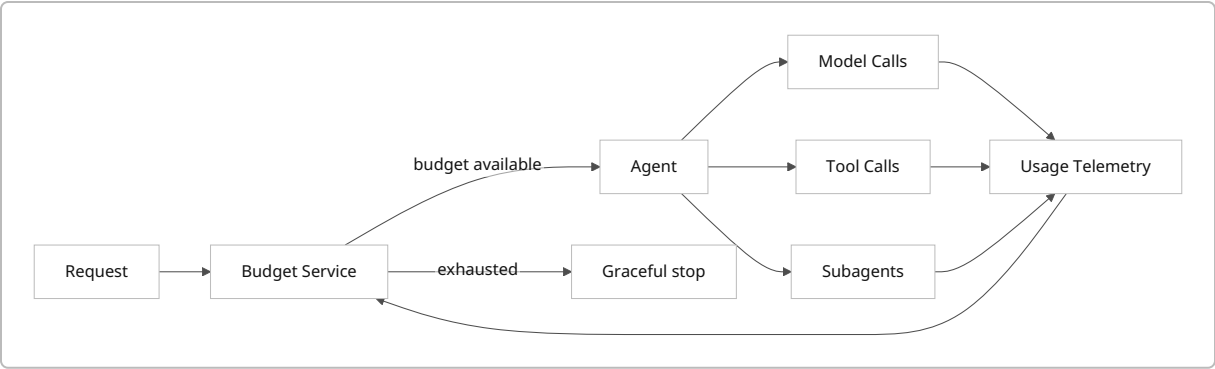
Sanitized incident traces:
  selectively extend

Evaluation dataset:
  deliberately curated and sanitized

Metrics:
  long enough for trend/SLO analysis
  no high-cardinality PII
```

LangSmith's base/extended trace tiers and indefinite dataset semantics make this separation particularly important. ⁴⁶

Recommended cost-governance architecture:



The budget record should track at least:

```
{
  "run_model_calls": 7,
  "thread_model_calls": 41,
  "run_tool_calls": 12,
  "run_input_tokens": 89432,
  "run_output_tokens": 7401,
  "run_model_cost_usd": 1.84,
  "run_tool_cost_usd": 0.37,
  "elapsed_seconds": 184,
  "subagents_created": 3,
  "max_parallelism_observed": 2
}
```

LangChain directly supplies model/tool call-count middleware, while LangSmith supplies token/cost observability and separate evaluator spending controls. A unified dollar/token/subagent/time budget remains an application-level governance layer worth building around those primitives. ⁹⁵

Suggested service-level objectives for mature agents should include reliability *and* quality:

Dimension	Example measure
Availability	successful terminal runs / started runs
Latency	P50/P95/P99 end-to-end and per model/tool
Queueing	P95 queue-wait time
Durability	recoverable runs after injected worker failure
Correctness	evaluator score on production sample
Trajectory quality	percentage satisfying tool/ordering policy

Dimension	Example measure
Safety	unauthorized side effects = zero
Human control	percentage of high-impact actions approved
Cost	dollars/task and tokens/task
Efficiency	tool calls/successful task
Recovery	mean time from transient fault to successful continuation
Security	policy denials, cross-tenant attempts, secret-detection rate

LangSmith experiment metrics include latency, token/cost, feedback, error, and time-to-first-token measures, while Prometheus/Sentry/OTel can cover runtime and service SLOs not specific to LLM evaluation. ⁹⁶

Implementation checklist and remaining research gaps

The following sequence gives you a practical path from a prototype Deep Agent to a hardened production harness.

Status	Implementation step	Definition of done
☐	Define state model	Explicit schemas for graph state, thread state, cross-thread memory, artifacts, and external-operation IDs
☐	Select persistent checkpointer	Production runs survive process/worker restart; no reliance on <code>InMemorySaver</code> for production durability. ⁴
☐	Establish stable <code>thread_id</code> policy	Every resumable job/conversation has tenant-safe stable thread identity
☐	Choose durability mode per workflow	<code>async</code> for normal work; <code>sync</code> where replay risk justifies checkpoint overhead; <code>exit</code> only for recomputable workflows. ⁵⁵
☐	Split large functions into durable nodes	Important boundaries correspond to recoverable graph steps
☐	Make side effects idempotent	Payment/write/delete/provision operations have operation IDs or reconciliation/compensation. ⁶
☐	Configure retry classification	Transient errors retry; policy/auth/programming failures do not. ¹⁵
☐	Configure async node timeouts	Long external calls have run/idle limits and explicit heartbeats where needed. ⁶²
☐	Add error handlers/compensation	Critical workflows have defined failure/rollback states. ⁶⁴

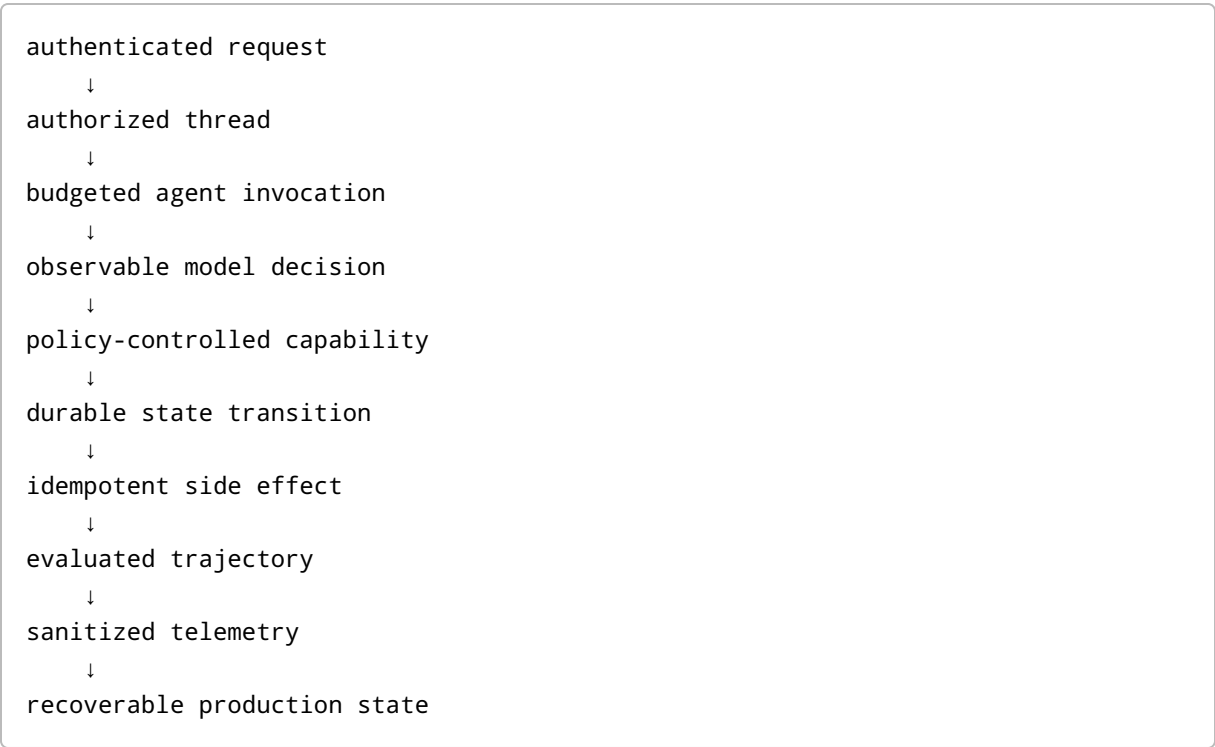
Status	Implementation step	Definition of done
☐	Build durable HITL	High-impact tool calls pause before execution and resume through the same thread. ⁸⁷
☐	Harden approvals	Reviewer sees exact operation, target, arguments, impact, and reason
☐	Avoid host shell in production	Use a sandbox rather than <code>LocalShellBackend</code> for untrusted execution. ¹⁰
☐	Constrain sandbox	Network, filesystem, CPU, RAM, disk, runtime, process count, and credentials are independently bounded
☐	Harden filesystem	Use State/Store backends by default; use <code>virtual_mode=True</code> for controlled <code>FilesystemBackend</code> roots. ⁷⁸
☐	Add backend policy hooks	Enforce path/content/rate/audit rules independently of model reasoning. ⁸³
☐	Namespace memory by user	No implicit writable cross-user memory. ¹¹
☐	Make organization policy memory read-only	Only trusted application/admin flow mutates shared policy state. ⁹⁷
☐	Implement Agent Server AuthN/AuthZ	Every thread/run/store/resource request is scoped to authenticated identity. ⁷⁵
☐	Reauthorize within tools	Tool side effects validate tenant/user permissions server-side
☐	Enable LangSmith tracing	Model/tool/subagent/run hierarchy visible and correlated with production IDs. ⁹⁸
☐	Add OTel correlation	Trace IDs propagate across agent → API → database → tool services. ⁹⁹
☐	Add Prometheus operational metrics	Queue, latency, retry, failure, checkpoint, token, and budget aggregates exist
☐	Add application error tracking	Agent exceptions correlate with surrounding service failures through Sentry/APM or equivalent. ¹⁰⁰
☐	Establish PII policy	Sensitive inputs/outputs are redacted before tracing or tracing is disabled for protected requests. ¹³
☐	Establish trace retention policy	Base vs extended retention selected intentionally. ¹⁰¹
☐	Establish dataset privacy policy	Production examples are sanitized before indefinite/long-lived evaluation storage. ¹⁰²

Status	Implementation step	Definition of done
☐	Track token and cost usage	Model/provider metadata produces correct per-run accounting. ¹⁰³
☐	Enforce run/thread call limits	Model/tool calls have bounded budgets. ⁸⁸
☐	Build total budget service	Dollar/token/time/subagent/concurrency limits are enforced before new operations
☐	Create golden offline dataset	Important use cases and historic failures exist as regression examples
☐	Add trajectory tests	Critical tool sequences are evaluated using strict/subset/unordered semantics where appropriate. ¹⁰⁴
☐	Add LLM judges	Subjective correctness/quality criteria have versioned rubrics. ¹⁰⁵
☐	Align judges with humans	Human corrections periodically test/calibrate automated evaluators. ¹⁰⁶
☐	Add online evaluators	Representative production traffic receives automated scoring within spend limits. ¹⁰⁷
☐	Build annotation workflow	Negative feedback, evaluator failures, and sampled traffic route to human queues. ¹⁰⁸
☐	Create security adversarial suite	Prompt injection, RAG injection, memory poisoning, cross-tenant access, shell abuse, exfiltration, and approval bypass are tested
☐	Perform crash/fault injection	Kill workers/processes and inject API/database failures while verifying safe recovery
☐	Test duplicate execution	Every consequential tool survives repeated execution without duplicate business effect
☐	Test stale resume	Old interrupted threads cannot execute changed operations under stale approval
☐	Version prompts/tools/graph/state	Trace metadata identifies exactly which runtime configuration generated behavior
☐	Define deployment migration rules	Existing checkpoints remain compatible or receive explicit state migration
☐	Define incident runbooks	Operators know how to cancel, inspect, resume, compensate, or quarantine a run

The implementation target is not merely:

```
agent.invoke(...)
```

It is:



That is the difference between an LLM demo and an **agent runtime**.

Several research areas are still worth mapping separately because they interact strongly with the three topics covered here.

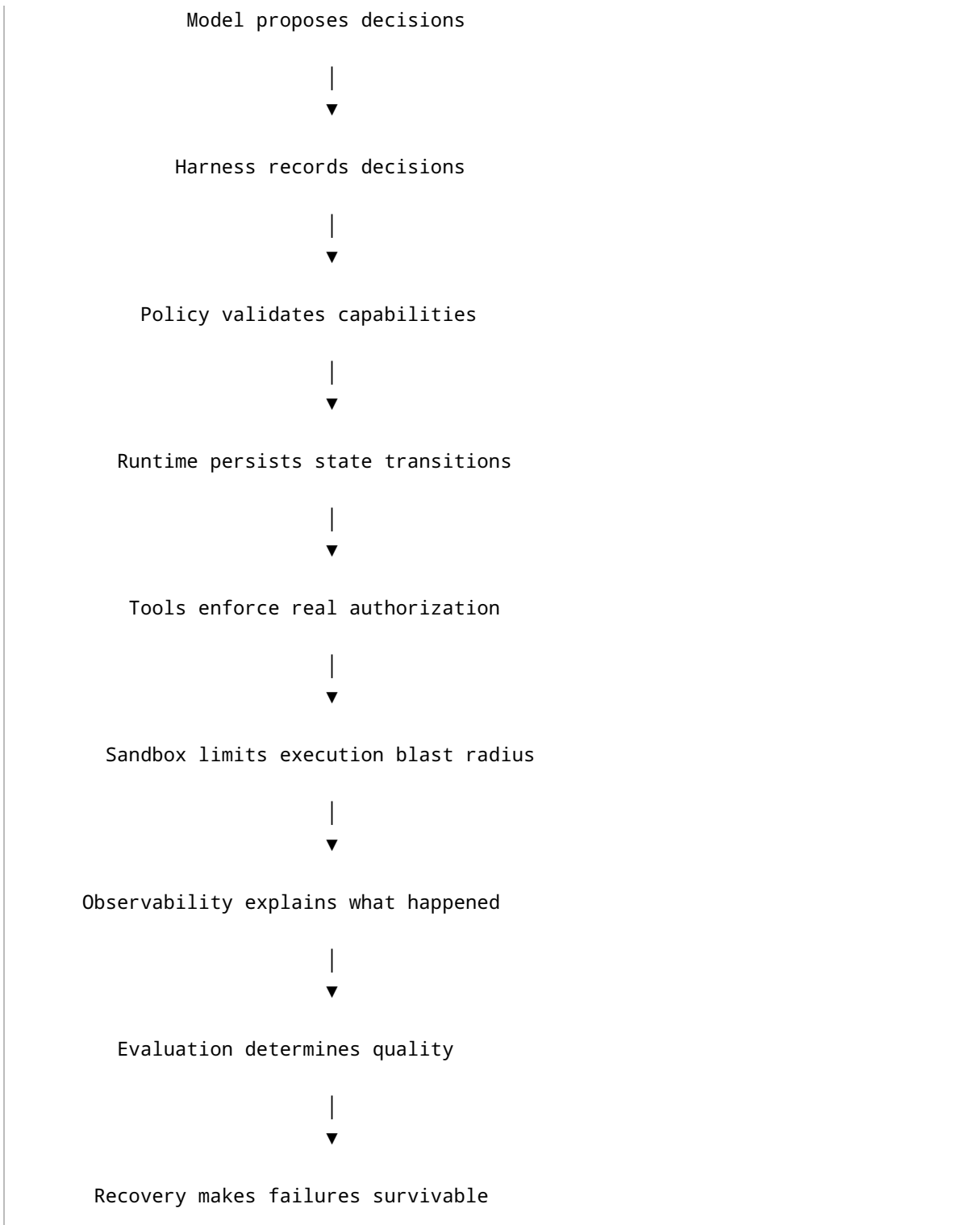
Remaining research area	Why it deserves a dedicated report
LangChain middleware internals and ordering	Middleware is becoming the primary policy/control plane for retries, PII, HITL, summarization, limits, filesystem, subagents, and tool behavior. The precise hook ordering and composition semantics determine security and observability correctness. ⁸⁶
MCP production security	Authentication, malicious servers, tool-schema drift, prompt injection through resources/tools, credential delegation, reconnect semantics, and server allowlisting create a separate trust-boundary problem.
Agent Server upgrade/migration semantics	Long-lived checkpoints may outlive application versions; graph/state/tool-schema migrations require explicit operational policy. LangGraph notes that resumed threads execute the currently compiled graph. ⁶

Remaining research area	Why it deserves a dedicated report
Distributed transactions and sagas	Truly consequential workflows need deeper treatment of outbox patterns, idempotency ledgers, reconciliation, external transaction IDs, and compensation beyond graph-level retries.
Subagent capability security	Deep Agents can delegate work, so capabilities, state, memory, budgets, and credentials should be scoped independently for supervisor and workers rather than automatically inherited. ¹⁰⁹
Context-security architecture	Context pruning, source trust labels, instruction/data separation, provenance, taint tracking, and tool-output filtering need to be integrated with prompt-injection defenses. Deep Agents explicitly acknowledges indirect-injection risk in RAG. ⁷¹
Agent artifact governance	Generated code/files/data need lineage, malware/content scanning, retention, ACLs, provenance, and lifecycle rules distinct from checkpoints and memory.
Formal budget middleware	LangChain provides model/tool call limits, but an integrated budget across dollars, tokens, time, subagents, tool charges, and concurrency is a valuable reusable harness component. ¹¹⁰
Chaos engineering for agents	A systematic test suite should inject model, tool, database, sandbox, network, cancellation, worker, and checkpoint failures and verify invariants automatically.
Security-specific trajectory evaluators	AgentEvals can test trajectories, but a reusable library of security invariants —“never read secret before external send,” “write requires prior approval,” “tenant B never appears in tenant A trajectory”—would significantly strengthen harness testing. ¹¹¹
OpenTelemetry semantic-convention strategy	GenAI semantic conventions are evolving; a stable internal telemetry schema can prevent observability backends from dictating application instrumentation. ¹¹²
Data lifecycle and right-to-delete	Checkpoints, Stores, LangSmith traces, evaluation datasets, artifacts, security audit logs, caches, and backups have different retention semantics and need a unified deletion/data-subject strategy. ⁴⁶

The most important final design principle is:

DO NOT TRUST THE MODEL
TO PROVIDE
ITS OWN CONTROLS





LangChain, LangGraph, Deep Agents, and LangSmith now provide most of the primitives needed to implement that architecture: middleware for limits, PII, retries, tool handling, and HITL; durable LangGraph checkpoints, interrupts, timeouts, error handlers, and replay; Deep Agents backends, permissions, filesystem, memory, subagents, and sandbox integration; Agent Server queues/workers; and LangSmith tracing, costs, evaluators, annotation workflows, retention controls, and OpenTelemetry integration. The

remaining engineering responsibility is to compose those primitives so that **security authorization, durable execution, observability, and evaluation remain independent of whatever the LLM happens to decide on a particular run.** ¹¹³

¹ ¹⁴ **Deep Agents overview - Docs by LangChain**

https://docs.langchain.com/oss/python/deepagents/overview?utm_source=chatgpt.com

² ⁹ ¹² ³⁷ ³⁸ ⁸⁶ ⁸⁸ ⁸⁹ ⁹³ ⁹⁵ ¹¹⁰ ¹¹³ **Prebuilt middleware - Docs by LangChain**

<https://docs.langchain.com/oss/python/langchain/middleware/built-in>

³ ¹⁸ ⁵⁴ ⁶⁸ **Agent Server - Docs by LangChain**

https://docs.langchain.com/langsmith/agent-server?utm_source=chatgpt.com

⁴ ¹⁶ ¹⁹ ⁵² **Persistence - Docs by LangChain**

https://docs.langchain.com/oss/python/langgraph/persistence?utm_source=chatgpt.com

⁵ ⁶ ⁵³ **Graph API overview - Docs by LangChain**

https://docs.langchain.com/oss/python/langgraph/graph-api?utm_source=chatgpt.com

⁷ ²⁴ ²⁷ ⁹⁹ **Trace with OpenTelemetry - Docs by LangChain**

https://docs.langchain.com/langsmith/trace-with-opentelemetry?utm_source=chatgpt.com

⁸ ⁴¹ ¹⁰⁴ ¹¹¹ **How to evaluate your agent with trajectory evaluations**

https://docs.langchain.com/langsmith/trajectory-evals?utm_source=chatgpt.com

¹⁰ ⁷⁷ **Backends - Docs by LangChain**

https://docs.langchain.com/oss/python/deepagents/backends?utm_source=chatgpt.com

¹¹ ⁹⁷ **Memory - Docs by LangChain**

https://docs.langchain.com/oss/python/deepagents/memory?utm_source=chatgpt.com

¹³ ²⁶ ⁴⁵ **Prevent logging of sensitive data in traces**

https://docs.langchain.com/langsmith/mask-inputs-outputs?utm_source=chatgpt.com

¹⁵ ⁶² ⁶³ ⁶⁴ ⁶⁵ **Fault tolerance - Docs by LangChain**

<https://docs.langchain.com/oss/python/langgraph/fault-tolerance>

¹⁷ ²¹ ⁴⁸ ⁹¹ ⁹⁸ **Observability concepts - Docs by LangChain**

https://docs.langchain.com/langsmith/observability-concepts?utm_source=chatgpt.com

²⁰ ⁸¹ ⁸² ¹⁰⁹ **Customize Deep Agents - Docs by LangChain**

https://docs.langchain.com/oss/python/deepagents/customization?utm_source=chatgpt.com

²² **Evaluate a complex agent - Docs by LangChain**

https://docs.langchain.com/langsmith/evaluate-complex-agent?utm_source=chatgpt.com

²³ ⁴⁹ **OpenTelemetry semantic conventions 1.44.0**

https://opentelemetry.io/docs/specs/semconv/?utm_source=chatgpt.com

²⁵ **Set a sampling rate for traces - Docs by LangChain**

https://docs.langchain.com/langsmith/sample-traces?utm_source=chatgpt.com

²⁸ **Redact sensitive data with the OpenTelemetry Gateway ...**

https://docs.langchain.com/langsmith/otel-gateway-trace-redaction?utm_source=chatgpt.com

29 112 **Gen AI**

https://opentelemetry.io/docs/specs/semconv/registry/attributes/gen-ai/?utm_source=chatgpt.com

30 31 50 **Understanding metric types**

https://prometheus.io/docs/tutorials/understanding_metric_types/?utm_source=chatgpt.com

32 59 60 87 **Interrupts - Docs by LangChain**

https://docs.langchain.com/oss/python/langgraph/interrupts?utm_source=chatgpt.com

33 **Streaming - Docs by LangChain**

https://docs.langchain.com/oss/python/langgraph/streaming?utm_source=chatgpt.com

34 69 **Monitor projects with dashboards - Docs by LangChain**

https://docs.langchain.com/langsmith/dashboards?utm_source=chatgpt.com

35 103 **Cost tracking - Docs by LangChain**

https://docs.langchain.com/langsmith/cost-tracking?utm_source=chatgpt.com

36 107 **Track and limit evaluator spend**

https://docs.langchain.com/langsmith/evaluator-spend?utm_source=chatgpt.com

39 43 **LangSmith Evaluation - Docs by LangChain**

https://docs.langchain.com/langsmith/evaluation?utm_source=chatgpt.com

40 **Evaluation types - Docs by LangChain**

https://docs.langchain.com/langsmith/evaluation-types?utm_source=chatgpt.com

42 **How to evaluate agents - Docs by LangChain**

https://docs.langchain.com/langsmith/evaluate-llm-application?utm_source=chatgpt.com

44 **Use annotation queues - Docs by LangChain**

https://docs.langchain.com/langsmith/annotation-queues?utm_source=chatgpt.com

46 101 **Usage and billing - Docs by LangChain**

https://docs.langchain.com/langsmith/usage-and-billing?utm_source=chatgpt.com

47 102 **Overview - Docs by LangChain**

https://docs.langchain.com/langsmith/administration-overview?utm_source=chatgpt.com

51 100 **Set Up Agent Tracing | Sentry for Python**

https://docs.sentry.io/platforms/python/agent-tracing/?utm_source=chatgpt.com

55 **Checkpointers - Docs by LangChain**

<https://docs.langchain.com/oss/python/langgraph/checkpointers>

56 **Thinking in LangGraph - Docs by LangChain**

https://docs.langchain.com/oss/python/langgraph/thinking-in-langgraph?utm_source=chatgpt.com

57 **Checkpointers - Docs by LangChain**

https://docs.langchain.com/oss/python/langgraph/checkpointers?utm_source=chatgpt.com

58 76 78 79 80 83 84 94 **Backends - Docs by LangChain**

<https://docs.langchain.com/oss/python/deepagents/backends>

61 **Customize Deep Agents - Docs by LangChain**

<https://docs.langchain.com/oss/python/deepagents/customization>

66 Enqueue concurrent - Docs by LangChain

https://docs.langchain.com/langsmith/enqueue-concurrent?utm_source=chatgpt.com

67 Configure Agent Server for scale - Docs by LangChain

https://docs.langchain.com/langsmith/agent-server-scale?utm_source=chatgpt.com

70 CAISI Issues Request for Information About Securing AI ...

https://www.nist.gov/news-events/news/2026/01/caisi-issues-request-information-about-securing-ai-agent-systems?utm_source=chatgpt.com

71 74 85 Retrieval Augmented Generation (RAG) with Deep Agents

https://docs.langchain.com/oss/python/deepagents/rag?utm_source=chatgpt.com

72 LLMRisks Archive - OWASP Gen AI Security Project

https://genai.owasp.org/llm-top-10/?utm_source=chatgpt.com

73 92 Insights into AI Agent Security from a Large-Scale Red- ...

https://www.nist.gov/blogs/caisi-research-blog/insights-ai-agent-security-large-scale-red-teaming-competition?utm_source=chatgpt.com

75 Authentication & access control - Docs by LangChain

https://docs.langchain.com/langsmith/auth?utm_source=chatgpt.com

90 Add encryption at rest

https://docs.langchain.com/langsmith/encryption?utm_source=chatgpt.com

96 How to fetch performance metrics for an experiment

https://docs.langchain.com/langsmith/fetch-perf-metrics-experiment?utm_source=chatgpt.com

105 Set up LLM-as-a-judge online evaluators

https://docs.langchain.com/langsmith/online-evaluations-llm-as-judge?utm_source=chatgpt.com

106 Improve LLM-as-judge evaluators using human feedback

https://docs.langchain.com/langsmith/improve-judge-evaluator-feedback?utm_source=chatgpt.com

108 Set up automation rules

https://docs.langchain.com/langsmith/rules?utm_source=chatgpt.com